

Sequence Analysis

1. Basics

2. Why do we compare sequences?

3. DotPlots

1. General Principle
2. Dotplot Parameters
3. Modern versions

4. Alignments

1. What is an Alignment?
2. Types
3. Quality of an Alignment
4. Substitution matrices
5. Gap Costs

5. Calculating Alignments

1. Dynamic programming
2. Needleman-Wunsch Algorithm
3. Semi-global Alignment
4. Smith-Waterman Algorithm

6. Analyzing Alignments

7. Pairwise Alignments in Python

8. Multiple sequence Alignments

1. Basics
2. ClustalW
3. Analyzing MSAs
4. Alternatives to ClustalW

Basics

- In bioinformatics as well as in biochemistry / biology and so on, sequences refer to chains of letters representing DNA/RNA or proteins
- In these chains each letter stands for one base or one amino acid in the corresponding macromolecule
- That means from a bioinformatics stand point sequences are nothing more than big strings
- Of course from a biological / biochemical standpoint these strings have specific meanings and certain letter combinations translate into specific functionalities

- When talking about the different aspects of sequence analysis, you have to keep in mind that normally all processes we discuss here are done with a sequence you have no / limited amount of knowledge about
- So that means you're mainly working with a newly discovered sequence and not with known sequences
- Furthermore all tools we will talk about were originally developed for working with protein sequences and later adjusted to work with nucleotide sequences as well
- We always have to keep in mind that all things that were developed are based upon things that could be detected, so there is a bias in the data however we can't change this bias

- When working in the field of sequence analysis you would start with a database search using FastA or BLAST
- Than you would create a Multiple sequence Alignment using tools like ClustalW
- Afterwards you would analyze this Multiple sequence Alignment to detect some sequences to use in pair-wise Alignments using algorithms like Needleman-Wunsch
- Last but not Least you would create a graphical comparison of your sequences using a Dotplot

- That means normally you start with the most complicated tool and then use less complicated ones afterwards
- During this course however we will start from the least complex topic and work our way up to the most complex one

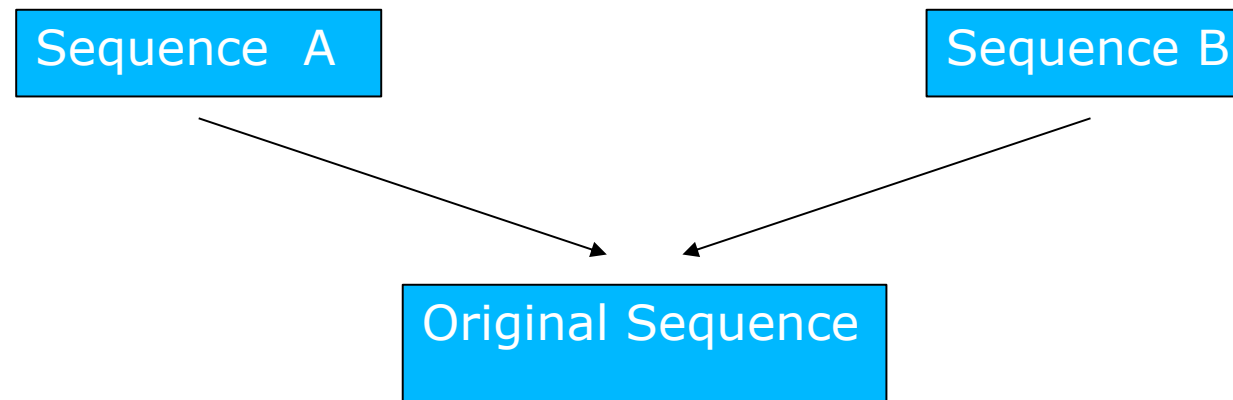
Why do we compare sequences?

- When we talk about comparing sequences we mean a letter by letter comparison of two different sequences
- During such a comparison we check whether these two sequences have the same letter at the same position or not
- For example comparing the sequence “VTA” to the sequence “LTA” means to compare V to L, T to T and A to A

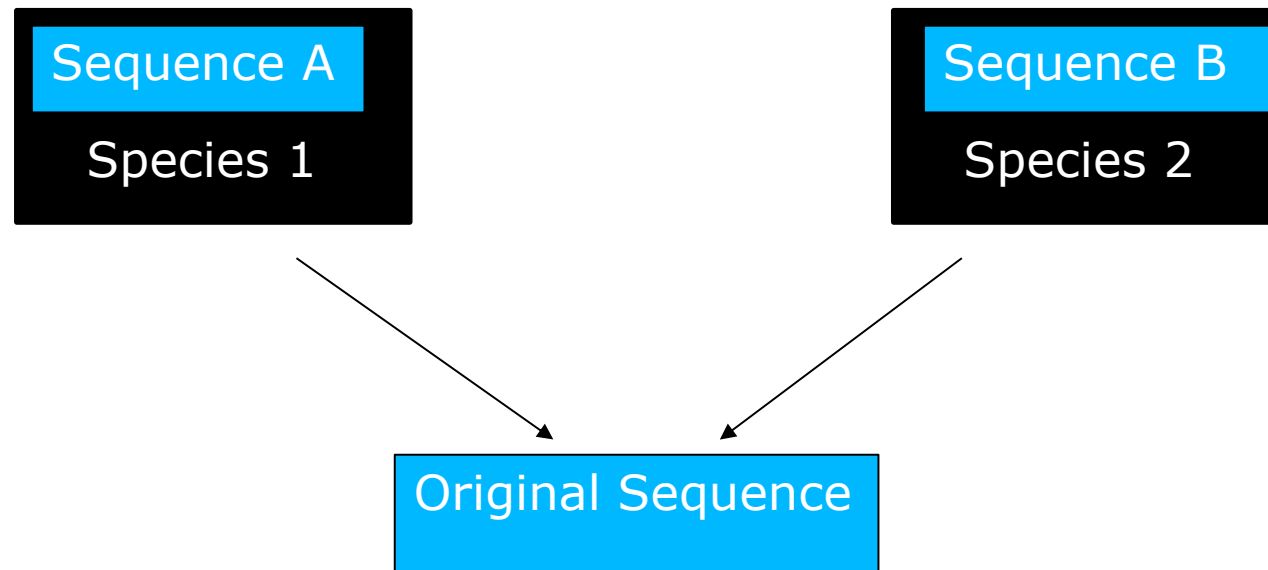
- We need to compare sequences with each other to determine the relations between them so that we can get more information about our sequence
- If we find a good documented sequence where the functionality behind it is known and we see that it is related to our sequence we can make guesses about the functionality of our sequence based on this relation
- There are three essential terms which describe these relations between sequences:
 - Homology
 - Orthology
 - Paralogy

- While these terms (see next slides) are defined in terms of genes, they of course still apply to proteins as well, since these genes are later translated into proteins
- That means if two proteins are related to each other, the corresponding genes are also related to each other and vice versa

Homolog: A gene related to a second gene by descent from a common ancestral DNA sequence. The term, homolog, may apply to the relationship between genes separated by the event of speciation (see ortholog) or to the relationship between genes separated by the event of genetic duplication (see paralog).



Ortholog: Orthologs are genes in different species that evolved from a common ancestral gene by speciation. Normally, orthologs retain the same function in the course of evolution. Identification of orthologs is critical for reliable prediction of gene function in newly sequenced genomes.



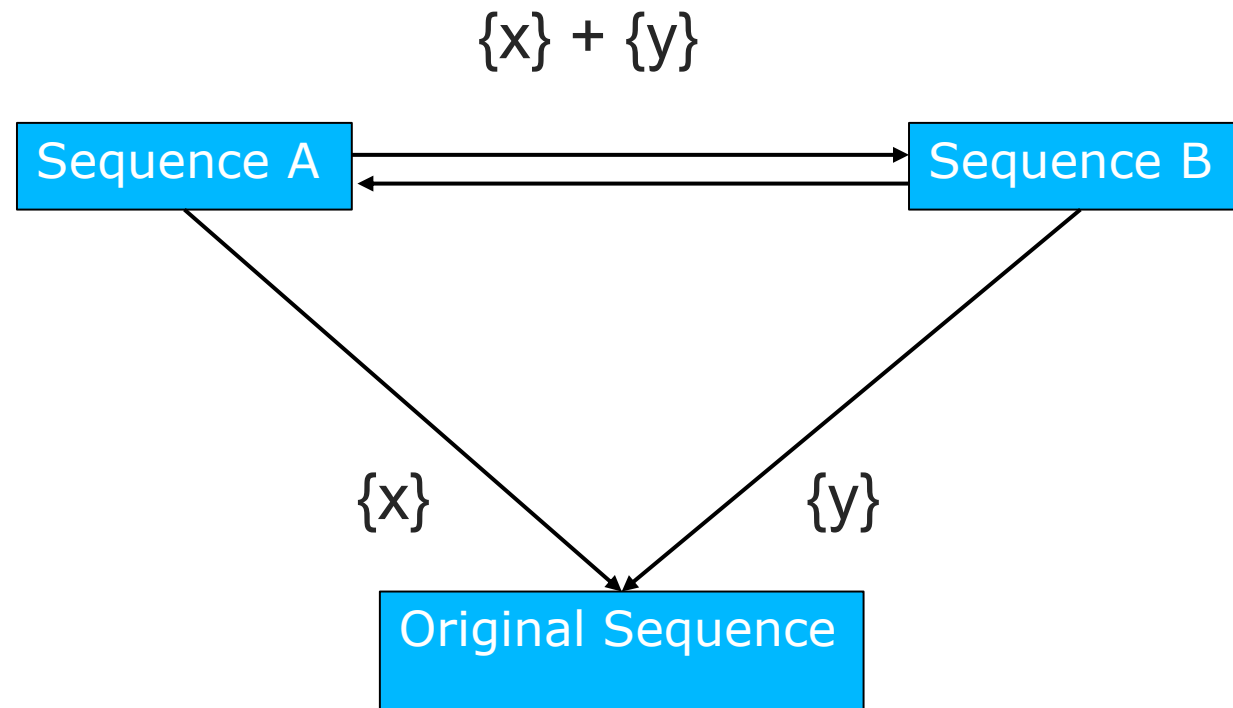
Speciation: Speciation is the origin of a new species capable of making a living in a new way from the species from which it arose. As part of this process it has also acquired some barrier to genetic exchange with the parent species.

Paralog Paralogs are genes related by duplication within a genome of one species. Orthologs retain the same function in the course of evolution, whereas paralogs evolve new functions, even if these are related to the original one.

Example : Hemoglobin, Myoglobin

- As soon as we know two sequences are homolog to each other, we want to know the grade of relationship between them
- This grade can be measured with the help of the evolutionary distance of our sequences
- As we will see, there are different ways of calculating this evolutionary distance and we need to keep in mind that all of them are just abstractions of reality, since we can not observe the process of evolution and get absolute measures

- In general the evolutionary distance between two sequences A and B can be defined as sum of the distances of sequence A to the original sequence and of sequence B to the original sequence:



- When talking about homology in regards to protein sequences we need to differentiate between two essential terms:
 1. Identity
 2. Similarity
- Identity refers to the percentage of identical amino acids when comparing two sequences with each other
 - ▶ For example if we have two sequences of 100 amino acids each and in 90 positions these sequences have the same letter they have an identity of 90%

- Sequence similarity however refers to the percentage of similar amino acids when comparing two sequences
- Amino acids are recognized as similar if they are the product of conservative mutations [2]
- In general amino acids are seen as similar if the physicochemical distance between them is relatively small
- However there are many ways of determining this physicochemical distance [3]
- The two most commonly used measures are the ones by Graham [4] and Miyate et al. [5]

- For general purposes we can use the different classes defined in the table below as a measure of similarity between amino acids

Class	Amino acids	Letters
Aliphatic	Glycine, Alanine, Valine, Leucine, Isoleucine	G, A, V, L, I
Hydroxyl or sulfur-containing	Serine, Cysteine, Threonine, Methionine	S, C, T, M
Cyclic	Proline	P
Aromatic	Phenylalanine, Tyrosine, Tryptophane	F, Y, W
Basic	Histidine, Lysine, Arginine	H, K, R
Acidic and their amides	Aspartate, Glutamate, Asparagine, Glutamine	D, E, N, Q

DotPlots

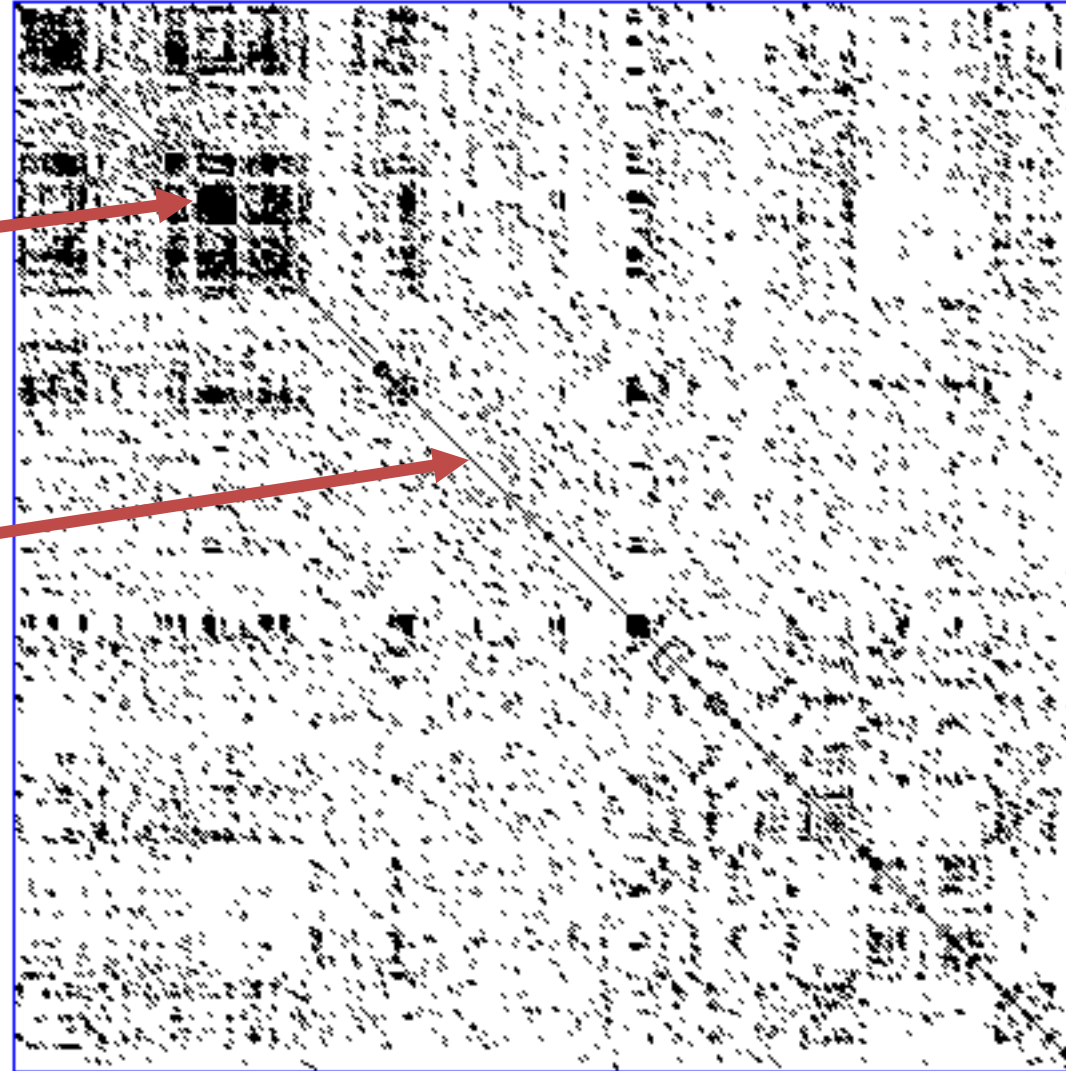
A **DotPlot** is a graphical method in bioinformatics to compare biological sequences with another (or a sequence with itself). Both sequences are applied to the horizontal and vertical (top and left) axis of a diagram and matches between a column and a row are marked with a point (line) at the specific intersection

	V	T	G	V	I	L	G
V	X			X			
T		X					
G			X				X
V	X			X			
I					X		
L						X	
G			X				X

	V	T	G	V	I	L	G
V	X			X			
T		X					
G			X				X
V	X			X			
T		X					
G			X				X
V	X			X			

repeated letters

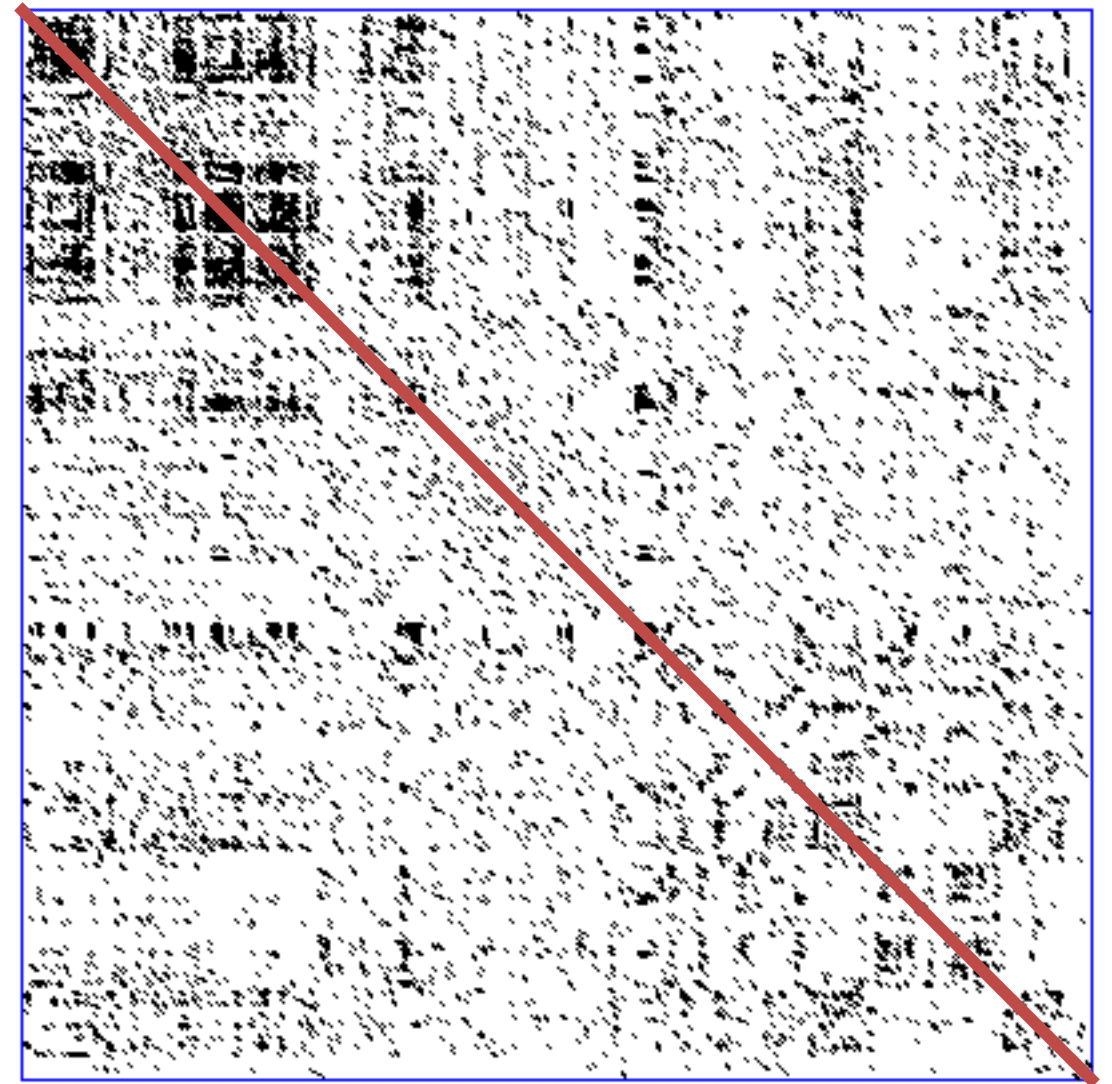
Main diagonal



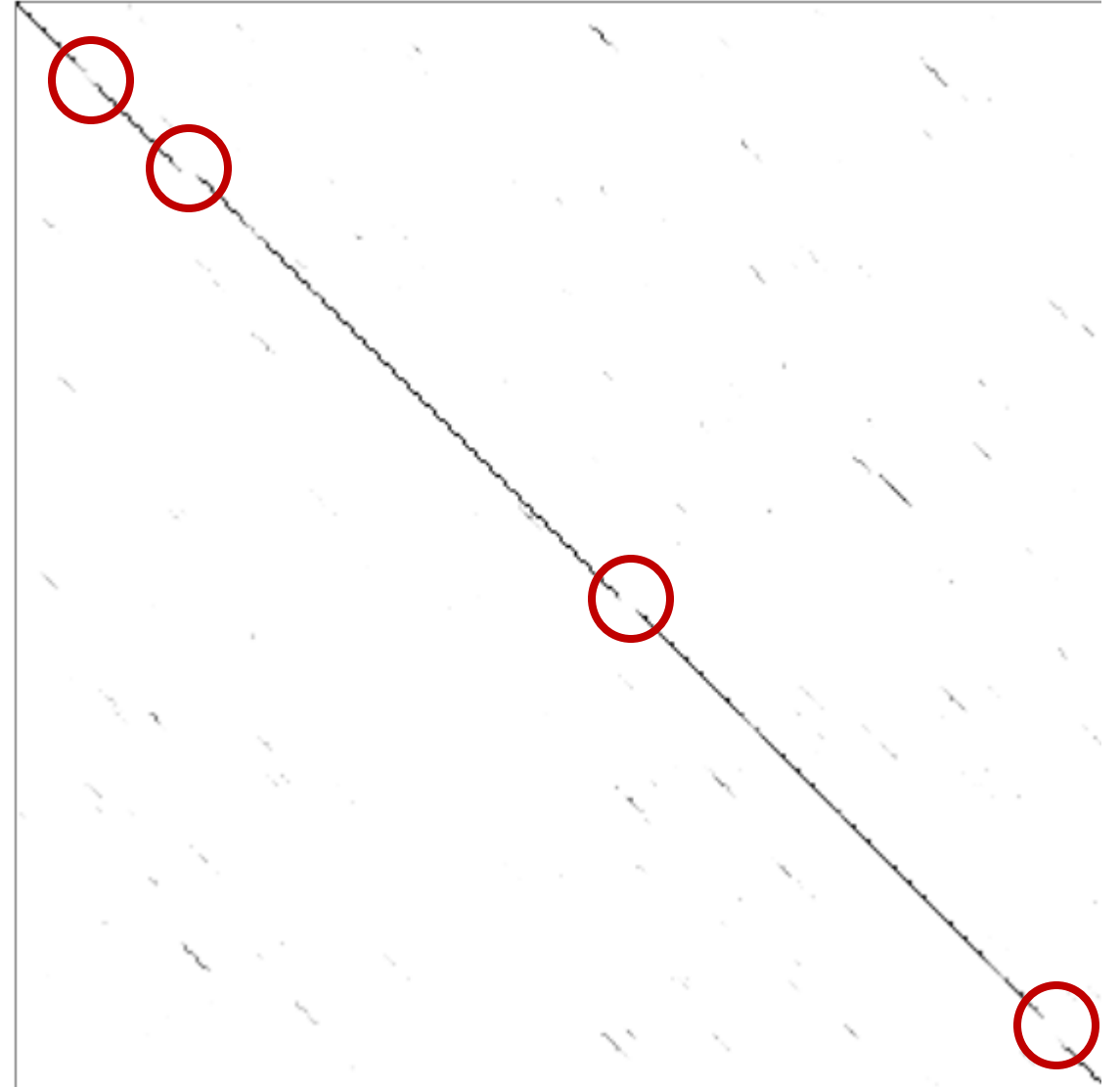
DotPlot of a Zinc-finger motif with itself

- To interpret a DotPlot you need to understand the meaning of different things
- Long diagonal lines in a DotPlot refer to long stretches of identical (similar) letters in both sequences
 - The most important diagonal is the main diagonal going from the top left corner to the bottom right corner, since it represents the letter by letter comparison of both sequences
- Breaks in these diagonals indicate “gaps”, that means regions where one sequence does not match the other sequence
 - This is mainly True if the diagonal continues afterwards
- Multiple parallel diagonals refer to repeating parts of the sequences
- Big blocks of Dots refer to “low complexity regions”, e.g. poly Alanine (AAAAAA)

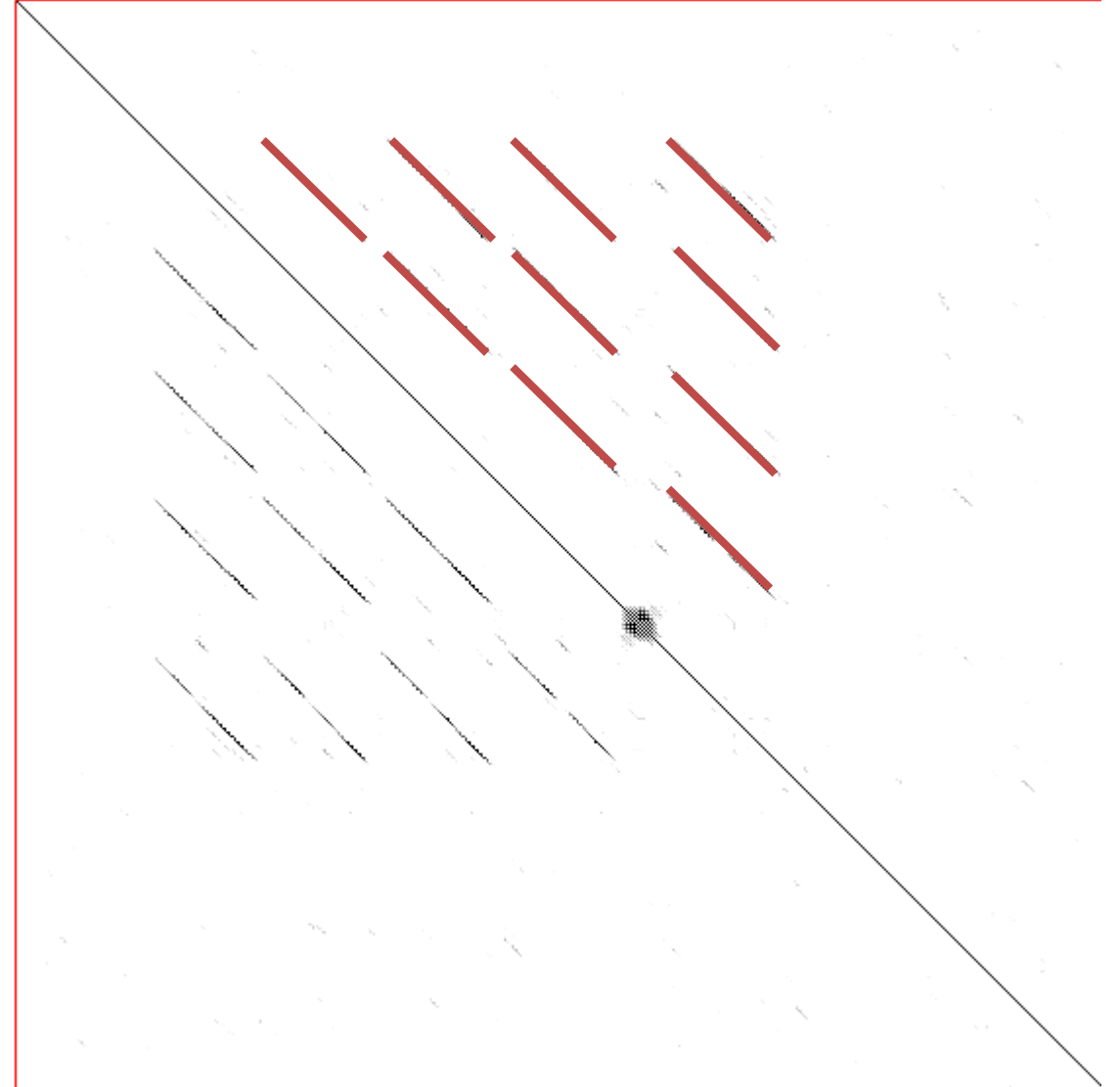
- In the plot on the right side we see a big ongoing main diagonal
- This is to be expected since we compare one sequence to itself, that means we have the exact same letter order on both axes here



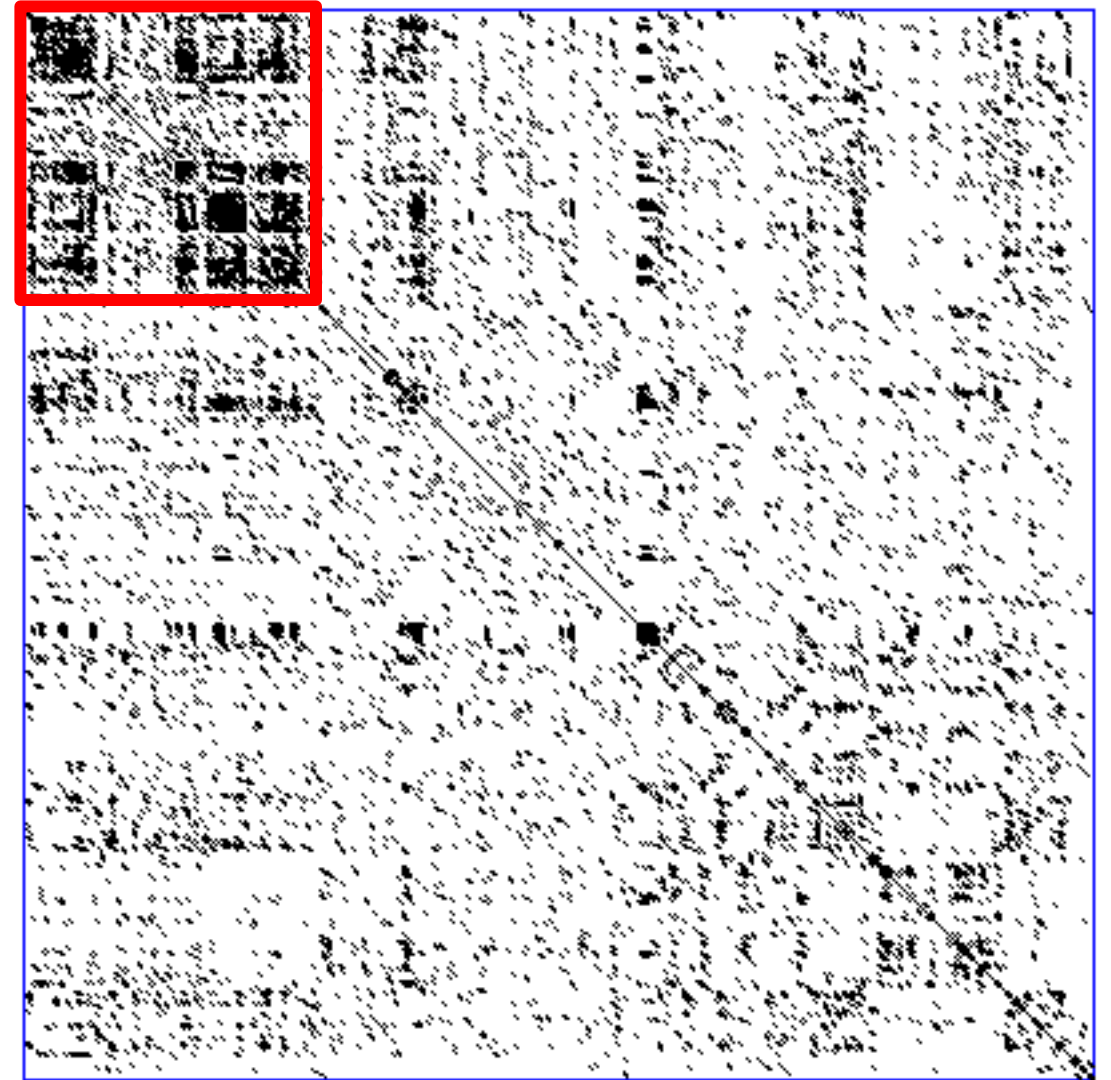
- In the plot on the right side we can again see a big main diagonal
- However this time it's interrupted at multiple positions (red circles)
- These interruptions represent gaps in either one of the sequences



- The plot on the right side shows a protein that contains multiple copies of a Kringle domain
- As you can see aside of the main diagonal there are many additional diagonals that represent the repeated domains (red lines)



- In the plot on the right side we can see a big square region in the top left corner with many dots and smaller squares that are nearly completely black
- This refers to a low complexity region, that means the same letter is repeated over and over again in this part of our sequence



- We discussed the general principle of creating DotPlots by combining sequences letter by letter earlier
- However scientists learned that sequences can be related to each other even if not all letters in them are identical
- In most cases only a certain amount of letters needs to be identical, while others can differ from each other
- To account for this biological background we can adjust DotPlots by using two very specific parameters

- The first parameter is called Windowsize W and tells the DotPlot how many letters of both sequences should be compared at once:
 - ▶ Our Dotplot now shows only matching substrings with the length W
 - ▶ That means we compare substrings of length W , e.g. 4 letters, with each other
 - ▶ If all letters in our window W are identical we then draw a diagonal line spanning all four fields that were compared using this parameter

- Looking back at the earlier example we had the sequence VTGVILG and by applying our window size to it we would get the following windows for comparison:
 - VTGV, TGVI, GVIL, VILG
- As you can see these windows are overlapping
- In a similar manner the Sequence VTGVTGV produces the windows:
 - VTGV, TGVT, GVTG, VTGV
- Now these windows are compared to each other, step by step

- The comparisons are as follows:
 - ▶ VTGV (seq1) against VTGV (seq2)
 - ▶ TGVI (seq1) against VTGV (seq2)
 - ▶ GVIL (seq1) against VTGV (seq2)
 - ▶ VILG (seq1) against VTGV (seq2)
 - ▶ VTGV (seq1) against TGVT (seq2)
 - ▶ TGVI (seq1) against TGVT (seq2)
 - ▶ GVIL (seq1) against TGVT (seq2)
 - ▶ VILG (seq1) against TGVT (seq2)
 - ▶ and so on
- Whenever the two compared windows match one of two types of markers can be set:
 - ▶ a line is drawn spanning the two compared windows
 - ▶ a dot at the central letter of both windows

- If we think back to our second example we saw earlier we see that we have two regions where we have the exact same four letters after each other
- By using the parameter W with a value of 4 the DotPlot would then look a bit different

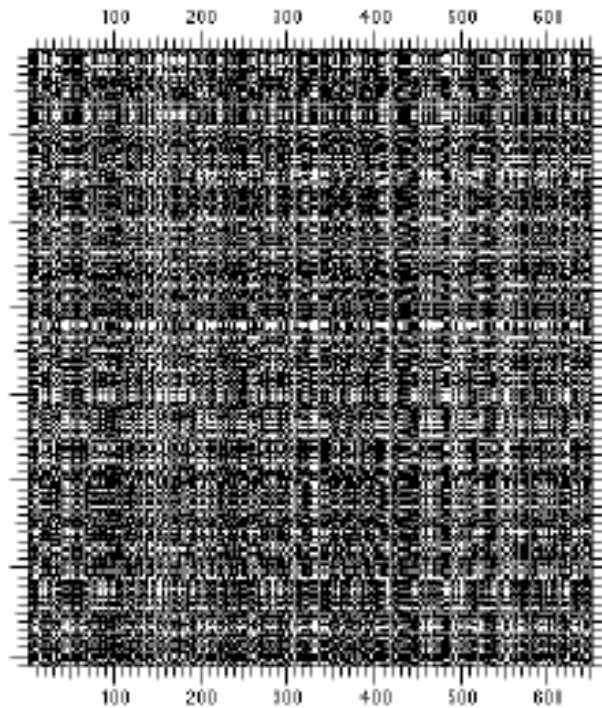
	V	T	G	V	I	L	G
V	X			X			
T		X					
G			X				X
V	X			X			
T		X					
G			X				X
V	X			X			

- The second parameter that was introduced is called Allowed Errors E or S:
 - ▶ Number of Errors, so called mismatches allowed within the Window size W
 - ▶ This parameter can only be used in combination with W
 - ▶ A window size of 4 with Allowed Errors of 1 means that one in four compared letters can differ between the two sequences in our Plot

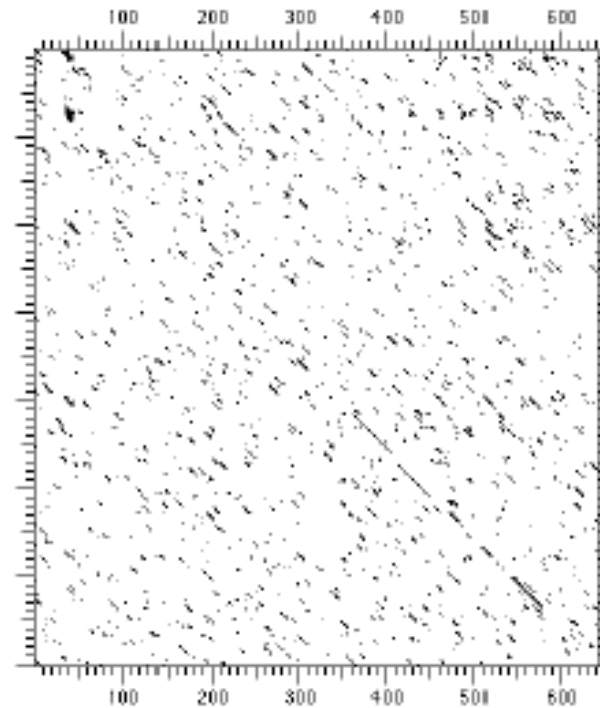
- We can show how the parameter E / S changes our plot if we look again at our second example from earlies
- By using the a window size of 4 and the parameter E with a value of 1, the DotPlot would again look a bit different
- We can see that the upper diagonal is now elongated by one square since the comparison of TGV I and TGV T is also accepted

	V	T	G	V	I	L	G
V	X			X			
T		X					
G			X				X
V	X			X			
T		X					
G			X				X
V	X			X			

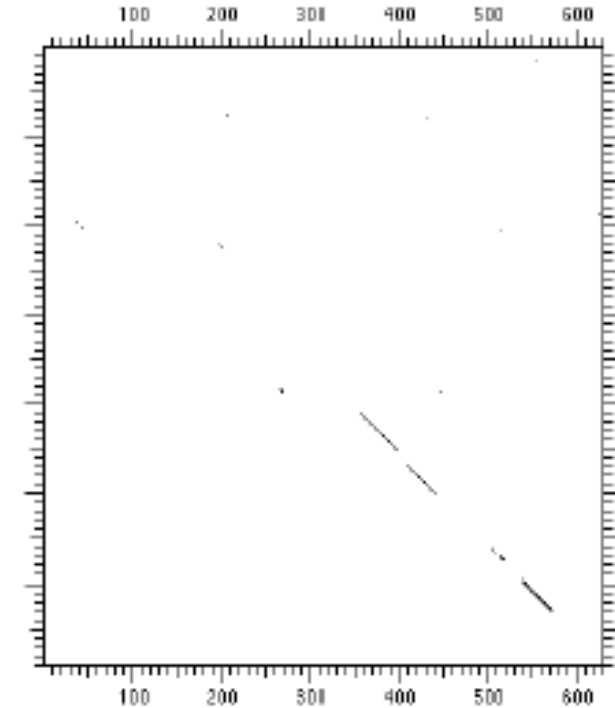
- With the help of these parameters we are able to reduce the “Noise” in our plots and thus we can lay the focus on the important parts of our comparison:



$$w = 1, s = 1$$



$$w = 11, s = 7$$



$$w = 23, s = 15$$

- After the introduction of the parameters W and E , DotPlots were adjusted even further to be able to model the relationship between sequences even better
- Instead of using absolute comparisons, e.g. V matched with V, modern DotPlots take into account that amino acids can be similar to each other even if they are not exactly the same
- To model this DotPlots use so called Substitution matrices, which we will discuss later in detail, that contain positive and negative values for all possible comparisons between letters of our alphabets

- When a program compares two substrings with a specific window size, the values for the different comparisons are looked up in the matrix and added together
 - For example when comparing TGV I and TGV T, we add up the values for T/T, G/G, V/V and I/T
- If the so calculated sum is above a certain positive threshold the program accepts this comparison and draws a diagonal line in the corresponding plot

Parameter	Protein	DNA	RNA
Window size	10-15	10-20	10-20
Allowed Errors	1-2	1-3	1-3
Threshold	23-30	23-30	23-30

- The Table above shows the most commonly used values for the three parameters of Dotplots
- Keep in mind that these values can be seen as start values and depending on your result you might need to adjust them

Examples A0A5N3XQ52 / A0A9D3NLI7

Examples B4DTL8 / B3JI28

Examples KR855596 / KR855587

- Use the fasta-files KR855587.fasta and KR855596.fasta to create some Dotplots using the Dotmatcher Tool of the EMBL
(https://www.ebi.ac.uk/Tools/seqstats/emboss_dotmatcher/)
- Use the combinations of window size and threshold shown in the table below for your plots
- Which combination produces the nicest result in your opinion?

Window size	Threshold	Window size	Threshold
10	30	25	30
10	45	25	40
15	35	30	45
20	23		

- You have the four fasta-files M3Z3I3.fasta, Q96PZ0.fasta, A0A5N3XQ52.fasta and A0A9D3NLI7.fasta
- All contain a TRUD-domain somewhere in their sequence
- Use Dotplots to find out where this domain could be in the different sequences
- Create 6 Dotplots in total, so that you can compare all sequences with each other
- Afterwards look up the sequences in the UniProt (the names of the files represent UniProt identifiers) and determine the correct position of the TRUD domain in them

- Use the two sequences P41221.fasta and P56704.fasta to create DotPlots with the help of Dotlet (<https://dotlet.vital-it.ch>)
- Use the combinations of window sizes and matrices shown below and try to figure out the best combinations of the two scales to get nice looking plots

Matrix	Window size	Matrix	Window size
BLOSUM62	15	BLOSUM80	10
BLOSUM45	15	BLOSUM62	20
BLOSUM80	15	BLOSUM45	20
BLOSUM62	10	BLOSUM80	20
BLOSUM45	10		

Alignments

- A way to compare two (or more) sequences with each other based on the amino acids / nucleotides in them
- An abstract measure of similarity for sequences
- From a bioinformatic standpoint it's just a way to write letters of two or more sequences below each other and compare them with each other
- While creating an alignment of two sequences there are exactly three possibilities we can encounter:
 - Letter – Letter (equal letters, but also different letters)
 - Letter – Nothing
 - Nothing – Letter

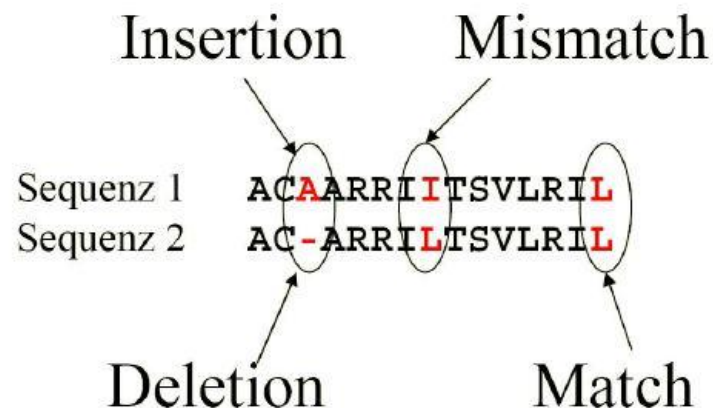
- For example aligning S1 = “ABC” with S2 = “ACE” could produce the following Alignment:

A - B C

A C - E

- Each letter of our sequences can be aligned only once either to nothing or another letter

- Down below you can see a short alignment of two sequences
- The different combinations in an alignment are referred to different terms as you can see and they represent different evolutionary causes:
 - ▶ A gap (-) models an INDEL (Insertion or deletion)
 - ▶ A mismatch represents a point mutation
 - ▶ A match shows conservation



- As mentioned earlier we can align two or more sequences at the same time
- When we use two sequences we talk about pairwise Alignments
- As soon as we have more than two sequences we talk about Multiple Sequence Alignments (MSAs)
- We will start by discussing pairwise Alignments

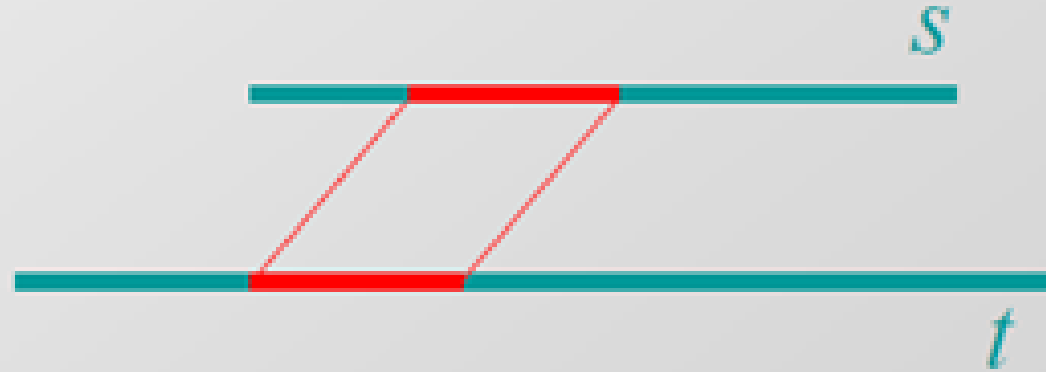
- There are several different types of alignments we can calculate to compare two different sequences with each other
- Each of them have their specific usage depending on what you want to know about the relation between your sequences
- In many cases you have to calculate different types to determine which fulfills your needs best

Global Alignment



- First and last letters of both sequences need to be aligned to each other, independently of them matching or not
- Used when both sequences are very close in length, maximum of around 20-30 letters
- Allows comparison of the whole sequences

Local Alignment



- Shows only the part of the sequences that match best with each other
- Used when you want to compare only parts of your sequences
- Allows the comparison of similar domains in sequences that differ strongly in other regions
- Can be problematic if your sequences are too different from each other



- The shorter sequence is aligned completely to a part of the longer sequence
- Used when one sequence is much shorter as the other
- Compares the shorter sequence to domains in the longer one
- Is especially useful to compare one-domain proteins with sequences of multi-domain proteins

- After we created an alignment we need to determine the quality of it
- There are two concepts called Score and “editing distance” that can be used to determine such a quality of an Alignment
- Both concepts use so called costs and there are costs for the occurrences of:
 - ▶ match
 - ▶ mismatch
 - ▶ gaps

- Match/Mismatch cost
 - A number taken from a substitution matrix or a fixed number
- Gap cost
 - A fixed number or calculated according to a function dependent on the length of the gap
- These costs are arbitrary but are oriented on the likelihood of occurrence of the three states
- That means they were calculated by comparing known sequences and determining how often specific amino acids mutate into each other


```
CAGCACTTGGATTCTCGG
CAGC-----G-T-----GG
VVVV-----V-V-----VV
```

- Matches get positive values
 - Mismatches and Gaps get (mostly) negative values
 - Example:
 - ▶ Costs: Match 1; Mismatch -1; Gap -2
 - ▶ 8 Matches + 10 Gaps + 0 Mismatches → $8 + (-20) + 0$
 - ▶ Score = -12
 - ▶ A higher score means a better alignment
- An optimal alignment has the maximal score

```
CAGCACTTGGATTCTCGG
CAGC-----G-T-----GG
VVVV-----V-V-----VV
```

- Matches get a cost of 0
 - Mismatches and gaps get a positive cost
 - Example:
 - ▶ Costs: Match 0; Mismatch 1; Gap 2
 - ▶ 8 Matches + 10 Gaps + 0 Mismatches → 0 + 20 + 0
 - ▶ Editing Distance = 20
 - ▶ A smaller editing distance means a better alignment
- An optimal alignment has the minimal editing distance

Match 1; Mismatch -1; Gap -2

HALLO

HELLO → 3

FAHREN

RAS_EN → -1

AUTOBAHN

RENNBAHN → 0

BASKETBALL

BAS_E_BALL → 4

Match 0; Mismatch 1; Gap 2

HALLO

HELLO → 1

FAHREN

RAS_EN → 4

AUTOBAHN

RENNBAHN → 4

BASKETBALL

BAS_E_BALL → 4

- Use the scoring schemes below to calculate the overall costs for the following four Alignments:

KRATZEN	FAULTIER	SCHAUKA_STEN
K_ATZEN	FAULPELZ	SCHAUFENSTER

UNITEDST__ATES

UNITEDMIRATES

- Use the following Costs:
 - ▶ Match 0; Mismatch 2; Gap 3
 - ▶ Match 2; Mismatch -2; Gap -3

- Calculate the optimal Alignments for the following three pairs of Strings:
- S1 = "TRACTOR" & S2 = "RACER"
- S1 = "WORLD" & S2 = "CROWD"
- S1 = "SKELETON" & S2 = "EXPLOSION"
- Use the following costs: Match= 5, Mismatch= -2 and Gap= -3

- For actual calculations no constant matching costs are used
- Instead Substitution matrices that contain costs for all possible letter combinations are used
- First substitution matrix: PAM1 developed Martha O. Dayhoff
- These matrices take the following into account:
 - ▶ Amino acids have different probabilities to mutate into one another
 - ▶ A Substitution matrix saves these values:
 - High positive number → probable mutation
 - High negative number → improbable mutation
 - A 0 → neutral mutation

PAM120 matrix

	C	S	T	P	A	G	N	D	E	Q	H	R	K	M	I	L	V	F	Y	W
C	9	-1	-3	-3	-5	-7	-7	-4	-6	-1	-8	-8	-2	-6	-1	-3	-8	-1	-12	
S	-1	3	2	1	1	0	-1	-2	-2	-3	-3	-4	-4	-5	-4	-5	-5	-4	-8	
T	-3	2	4	1	-1	-2	-2	-3	-4	-4	-5	-5	-4	-5	-4	-5	-5	-4	-8	
P	-3	1	1	6	-1	-2	-2	-3	-4	-4	-5	-5	-4	-5	-4	-5	-5	-4	-8	
A	-3	1	1	1	3	-1	-2	-2	-3	-4	-4	-5	-5	-4	-5	-5	-5	-4	-8	
G	-5	1	-1	-2	2	5	0	-1	-2	-3	-3	-4	-4	-5	-4	-5	-5	-4	-8	
N	-5	1	0	-2	0	0	4	0	-1	-2	-3	-4	-4	-5	-4	-5	-5	-4	-8	
D	-7	0	-1	-2	0	0	2	5	0	-1	-2	-3	-4	-4	-5	-4	-5	-4	-8	
E	-7	-1	-2	-1	0	-1	1	5	5	0	-1	-2	-3	-4	-4	-5	-4	-5	-8	
Q	-7	-2	-2	0	-1	-3	0	1	2	6	0	-1	-2	-3	-4	-4	-5	-4	-8	
H	-4	-2	-3	-1	-3	-4	2	0	-1	5	7	0	-1	-2	-3	-4	-4	-5	-8	
R	-4	-1	-2	-1	-3	-4	-1	-3	-3	1	1	6	0	-1	-2	-3	-4	-4	-8	
K	-7	-1	-1	-2	-2	-3	1	-1	-1	0	-2	2	5	0	-1	-2	-3	-4	-8	
M	-6	-2	-1	-3	-2	-4	-3	-4	-4	-1	-4	-1	0	8	0	-1	-2	-3	-8	
I	-3	-2	0	-3	-1	-4	-2	-3	-3	-3	-4	-2	-2	1	6	0	-1	-2	-8	
L	-7	-4	-3	-3	-3	-5	-4	-5	-4	-2	-3	-4	-4	3	1	5	0	-1	-8	
V	-2	-2	0	-2	0	-2	-3	-3	-3	-3	-3	-3	-4	1	3	1	5	0	-8	
F	-6	-3	-4	-5	-4	-5	-4	-7	-6	-6	-2	-4	-6	0	0	0	-3	8	-8	
Y	-1	-3	-3	-6	-4	-6	-2	-5	-4	-5	-1	-6	-6	-2	-2	-3	-3	4	8	
W	-8	-2	-6	-7	-7	-8	-5	-8	-8	-6	-5	1	-5	-7	-7	-5	-8	-1	12	

- On the diagonal (red rectangle) you can see the match costs
- All other numbers represent the costs for mismatches:
 - ▶ values near zero (blue circle) refer to similar amino acids
 - ▶ strongly negative values (green circle) refer to very different amino acids

- Stands for point accepted mutation
- Calculation of these matrices was based on 1572 observed mutations in closely related proteins
- The proteins to be studied were selected on the basis of having high similarity between each other
- That means the alignments included were required to display at least 85% identity
- As a result any aligned mismatches could thus be seen as the result of a single mutation event during the evolution process

- For each Alignment the number of observed mismatches were recorded sorted by the two amino acids involved
 - That means they simply counted how often each amino acid was aligned with another different amino acid
- These counts were used as entries below the main diagonal of the matrix A
- Since we can't define the direction of a mutation between two sequence, the matrix A is assumed to be symmetric, and thus entries of A above the main diagonal are identical to the ones below
- The entries along the diagonal of A do not correspond to mutations and can be left unfilled

- With the help of these counts Dayhoff et al. could also determine values for the frequency as well as mutability of each amino acid [7]
- Frequency
 - simply refers to the percentage of an amino acid being present in all sequences
 - is calculated by dividing the number of appearances N_{AA} of a specific amino acid by the number of all amino acids N_{total} in all sequences
- Mutability:
 - is a measurement to determine how often an amino acid mutates
 - is calculated by dividing the number of an amino acid present in a mutation $N_{mut,AA}$ by the number of appearances of it in all sequences N_{AA}

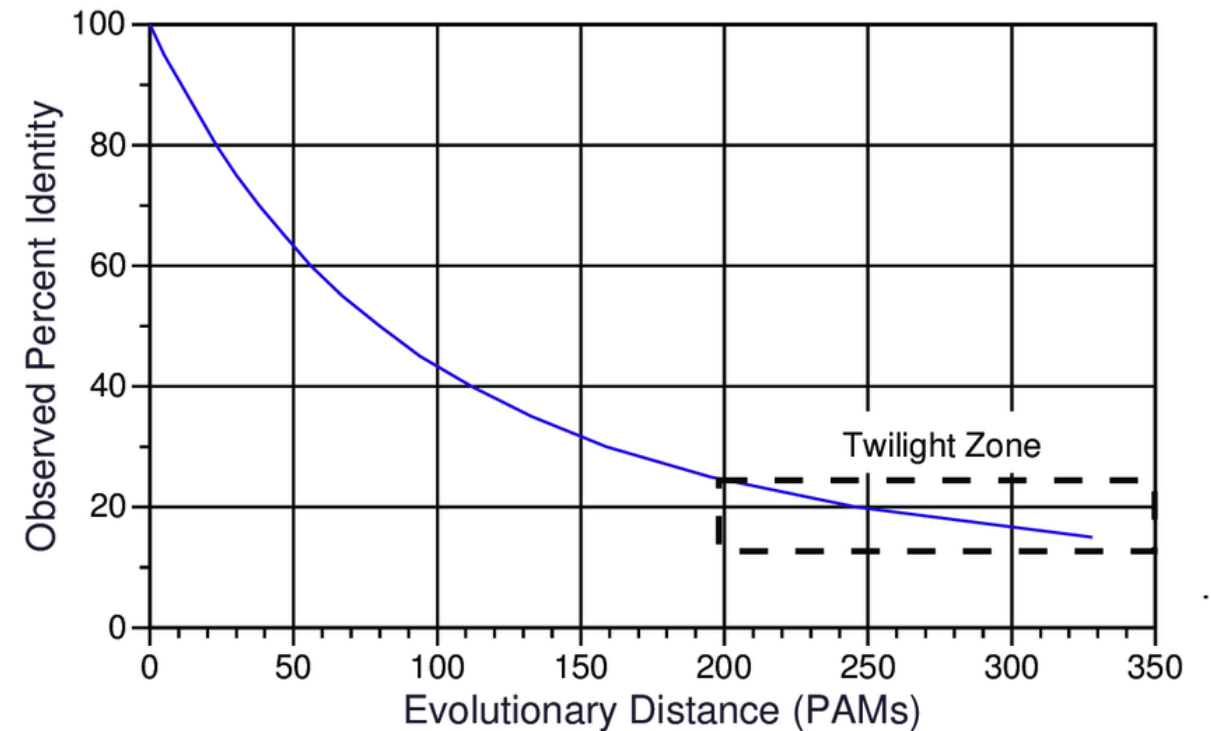
- With the help of the counts, the frequency and the mutability of the different amino acids a substitution matrix could be calculated
- The result of these calculations was the PAM1 matrix the predecessor of all PAM matrices
- Other PAM matrices are then calculated using an interpolation of the values of PAM1

- When using the PAM_n matrix to score an alignment of two proteins, the following assumption is made:
 - ▶ If these two proteins are related, the evolutionary interval separating them is the time taken for n point accepted mutations to occur per 100 amino acids
 - ▶ That means for PAM250 there were 250 mutations on 100 amino acids
 - ▶ This is of course possible because the same amino acid can be mutated more than once over the course of evolution

- When using PAM matrices for scoring sequence alignments the table and graph on the right side can be a good helper to determine which matrix fits your needs, depending on the identity of the sequences you want to compare

PAM	0	30	80	110	200	250
%identity	100	75	50	40	25	20

The Limits of Sequence Similarity



- Stands for BLOcks SUBstitution Matrix
- BLOSUM matrices are used to score alignments between evolutionarily divergent protein sequences
- They are based on local alignments instead of global alignments as used by PAM matrices
- By using the BLOCKS database, which contains sequence alignments for conserved protein domains they were able to get local Alignments without any gaps in them [10]

- From these local Alignments the relative frequencies of amino acids and their substitution probabilities were counted
- This was done by simply counting how often each amino acid appeared throughout all sequences and then how often it was involved in specific mutations
- That means the general process was very similar to the one used by Dayhoff for calculating PAM matrices

- From the counted numbers log-odds scores were calculated for all 190 possible combinations of the 20 canonic amino acids
- Several sets of BLOSUM matrices exist using different alignments as basis
- Just like PAM matrices they're named with numbers, e.g. BLOSUM62
- BLOSUM matrices with high numbers are designed for comparing closely related sequences, while those with low numbers are designed for comparing distant related sequences

- BLOSUM80 is used for closely related alignments, that means the sequences have a high percentage identity
- BLOSUM45 is used for more distantly related alignments, that means the sequences have a low percentage identity
- The numbers represent the percentage identity of the sequences used to calculate the corresponding matrix
- E.g. BLOSUM62 is the matrix built using sequences with $\geq 62\%$ identity

- When comparing BLOSUM45 and BLOSUM90 (see ABI_Blosum-Matrices.xlsx) there are several aspects that stand out
 1. BLOSUM90 contains a higher amount of strongly negative values (smaller -2)
 2. The match scores (diagonal) are higher in BLOSUM90
 3. The amount of probable mutations (bigger 0) outside the matches is higher in BLOSUM45
- What are the reasons:
 1. BLOSUM45 allows for more mismatches, since it was developed for less similar sequences
 2. BLOSUM90 forces the alignment into matches or gaps, to allow for a high identity

- When comparing PAM and BLOSUM matrices we can make use of the small table on the right
- As we can see the numbers follow a negative correlation
- So for very similar sequences we need BLOSUM matrices with high numbers and PAM matrices with low numbers

PAM	BLOSUM
PAM20	BLOSUM90
PAM40	BLOSUM80
PAM60	BLOSUM62
PAM80	BLOSUM50
PAM100	BLOSUM45

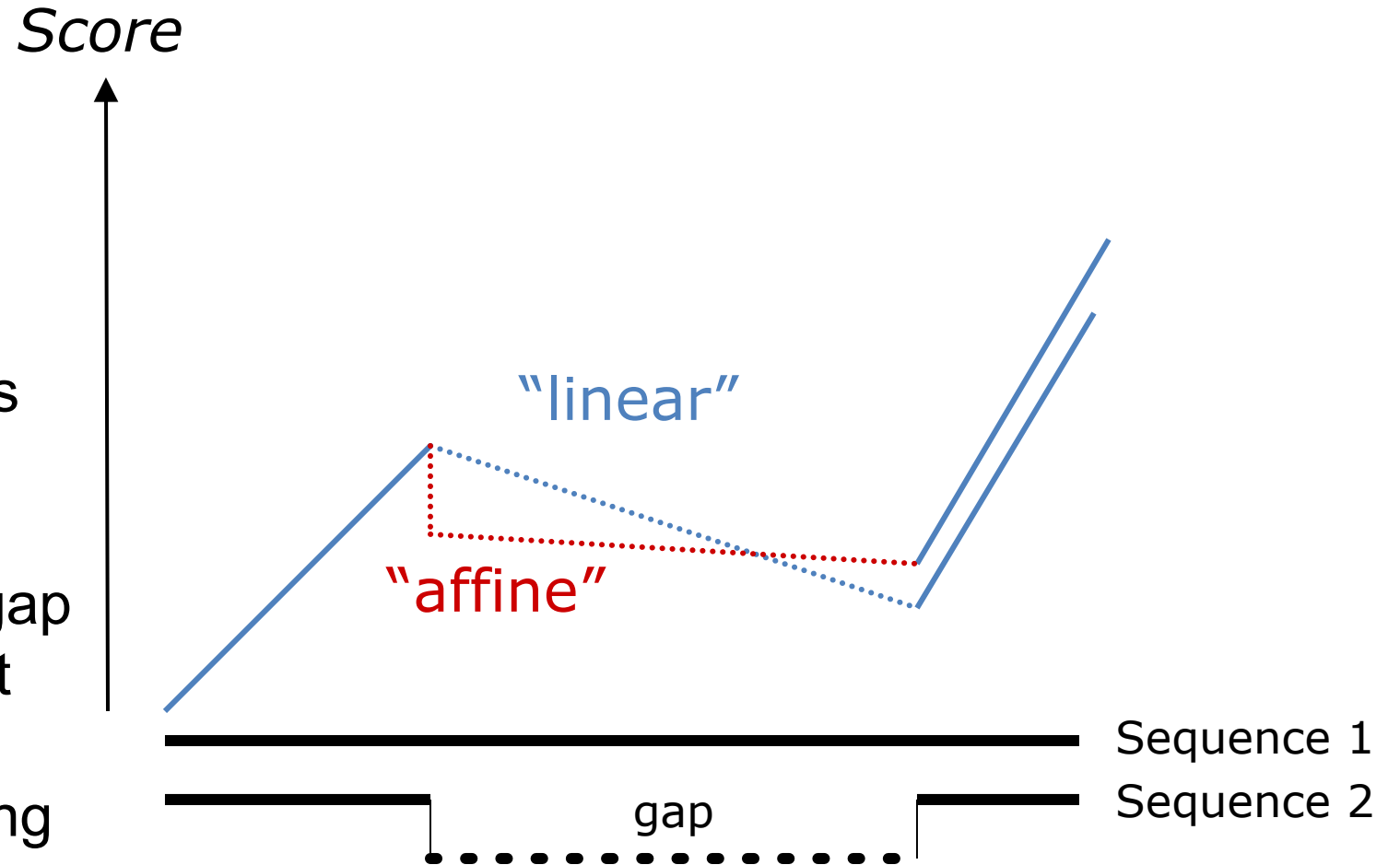
PAM	BLOSUM
To compare closely related sequences, PAM matrices with lower numbers are created	To compare closely related sequences, BLOSUM matrices with higher numbers are created
To compare distantly related proteins, PAM matrices with high numbers are created	To compare distantly related proteins, BLOSUM matrices with low numbers are created

- For your exercises you worked with so called linear gap costs:
 - ▶ Each letter that was aligned to a gap was assigned the exact same cost
 - ▶ The overall costs for all gaps was only dependent on the number of letters aligned to a gap
- In reality however so called affine gap costs are used instead which reproduce the natural behavior in a better way:
 - ▶ The first letter aligned to a gap is assigned a much higher cost
 - ▶ If the gap spans more than one letter, the remaining letters are assigned a much smaller cost
 - ▶ That means overall one long gap has a smaller cost in comparison to many small gaps

- Affine Gap Cost follows the formula:
 - $G(k) = a + b \cdot (k-1)$
- $G(k)$ is the cost for one specific gap
- k is the length of the Gap, that means the number of adjacent letters in one sequence that aren't aligned to letters in the second sequence
- “a” and “b” are pre-defined constants with “a” being the so-called gap-open cost and “b” being the so called gap-extension cost

- Simple Example using $a=-10$ and $b=-1$
- One gap with 3 letters would get an overall gap cost of:
 - $-10 + 2 \cdot -1$
 - The result would be -12
- Three gaps with one letter each would get an overall gap cost of:
 - $(-10) + (-10) + (-10)$
 - The result would be -30
- Since we want the highest possible score, one big gap is preferable here

- The diagram on the right shows how the score changes depending on the gap cost
- At the beginning the cost behaves identical for both types
- Over the gap, the score decreases more for the linear gap cost than for the affine gap cost
- At the end the alignment using the affine gap cost scores better



- There are at least two other proposed ways of calculating Gap costs
- One is the logarithmic Gap cost: $G(k) = a + c \ln(k)$
- Another one is the so-called log-affine Gap cost

Calculating Alignments

- When talking about alignments we need to keep in mind that only optimal alignments can be used for comparisons between our sequences
- So we need to figure out a way to find the best Alignment from all possible alignments two sequences could have
- The naive procedure would be to calculate all possible alignments between the sequences and choose the most optimal one(s)
- This is called the “brute force” algorithm

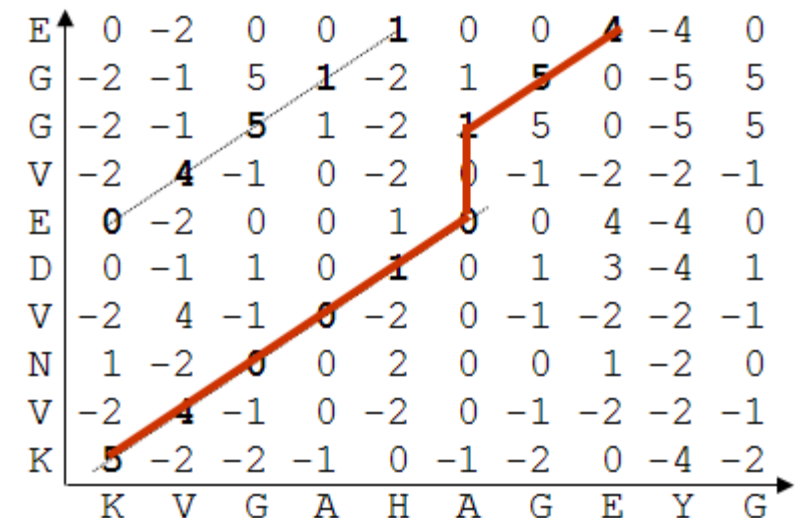
- There is a formula that allows us to calculate how many Alignments we would to create depending on the number of letters n and m in our two sequences
- By using this formula we can calculate the number of Alignments we need to create for two sequences with 10, 20 or 100 letters each (table)

$$\binom{n+m}{n} * \frac{(m+n)!}{(m!)^2} \sim \frac{2^{m+n}}{\sqrt{\pi * m}}$$

n = m	Enumeration
10	184,756
20	$1.4 * 10^{11}$
100	$9.0 * 10^{58}$

- For $n = 200$, even if each alignment would need only a fraction of a second, you have to calculate 10000 times the age of the universe
- ➔ Extremely high number of alignments (exponential growth with the length of the sequences)
- ➔ Clever methods for the calculation are necessary

- What proved to be the most effective when calculating alignments was to visualize all possible alignments as a NxM matrix with:
 - ▶ Sequence 1 being on the vertical axis
 - ▶ Sequence 2 being on the horizontal
- Each path through this matrix then represents one possible alignment and we have to find the one with the highest score



- To be able to find this path we use a recursive dynamic programming approach developed by Saul B. Needleman and Christian D. Wunsch in 1970 [11]
- Dynamic programming means to separate a big problem (the whole sequence) into smaller problems and then using the solutions of the smaller problems to find the optimal solution for the larger problem
- Recursion refers to calculating the solution of a problem by using the solutions of earlier problems

- As mentioned earlier Saul B. Needleman and Christian D. Wunsch developed an algorithm to calculate global Alignments with the help of a matrix [12]
- To honor the two developers the algorithm is called Needleman-Wunsch algorithm and it uses a dynamic programming approach using recursive matrix creation
- It is still widely used for finding the optimal global alignment, particularly when the quality of the global alignment is of the utmost importance

- In general the algorithm has two major parts:
 1. The creation of the Alignment matrix
 2. The search for the optimal way through the matrix using a back-tracking algorithm
- Each of the two steps reduces the amount of possible alignments drastically and thus reduces the calculation time immensely
- On the next slides we will go over the calculation process step by step while using small example sequences

- For our Example we want to find the optimal Alignment of sequence1 MGKP and sequence2 MGPKKP
- In the first step we need to create a grid as shown on the right side with
 - ▶ 5 columns, one for each letter in sequence1 + an additional empty one at the beginning
 - ▶ 7 rows, one for each letter in sequence2 + an additional empty one at the beginning

		M	G	K	P
M					
G					
P					
K					
K					
P					

- After the grid was generated we need to fill it with scores by going through it row by row
- The score for each cell (red) is calculated by using the scores of the neighboring cells to the left, top or top-left (diagonal) (green)
- Using this approach we calculate three potential scores for each field, but will save only the highest one

		M	G	K	P
M					
G					
P					
K					
K					
P					

- Since we need to start our calculation with at least one value, the top left corner of our grid is filled with a 0 in the first step
- Afterwards we can calculate the scores for the next cells using the neighbors
- Each of these three directions (coming from the left, the top or diagonal) has a different meaning in regards to an Alignment

		M	G	K	P
	0				
M					
G					
P					
K					
K					
P					

- The paths coming from the left or the top neighbor of a cell represent the introduction of a gap into our Alignment
 - ▶ The resulting score is than the score in the cell we start + the gap cost, -6 in our example
- The path coming from the diagonal represents Match/Mismatch in our Alignment
 - ▶ The resulting score is than the score in the cell we start + the Match/Mismatch score from PAM250

		M	G	K	P
	0				
M					
G					
P					
K					
K					
P					

- Now we can start filling our grid, by calculating the scores for the first row
- Since each cell in the first row only has left neighbors, the scores can be calculated by adding -6 every time we move to the right

		M	G	K	P
	0	-6	-12	-18	-24
M					
G					
P					
K					
K					
P					

- For the first column in our grid a similar rule applies, since all cells in it only have a neighbor at the top
- So it can also be filled by simply adding -6 for everytime we move on cell down
- The resulting grid now contains scores for the first row and first column, which can than be used for the next calculations

		M	G	K	P
	0	-6	-12	-18	-24
M	-6				
G	-12				
P	-18				
K	-24				
K	-30				
P	-36				

- Now the score for the cell that represents the Match M/M (red, PAM250 Score=6) has to be calculated
- To do so we have to calculate all three possibilities:
 - ▶ coming from the left: $-6 + (-6) = -12$
 - ▶ coming from the top: $-6 + (-6) = -12$
 - ▶ coming from diagonal: $0 + 6 = 6$

		M	G	K	P
	0	-6	-12	-18	-24
M	-6				
G	-12				
P	-18				
K	-24				
K	-30				
P	-36				

- Since our calculations showed that coming the diagonal way, leads to the highest score of 6 we will fill the cell with this value and we will save which direction was used to calculate this score as well (red arrow)

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6			
G	-12				
P	-18				
K	-24				
K	-30				
P	-36				

- Now we need to fill the next cell in the current line, which represents the Mismatch M/G (red, PAM250 Score=-3) by again calculating all three possible scores:
 - coming from the left: $6 + (-6) = 0$
 - coming from the top: $-12 + (-6) = -18$
 - coming from diagonal: $-6 + -3 = -9$

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6			
G	-12				
P	-18				
K	-24				
K	-30				
P	-36				

- This time the highest score was derived while coming from the left with a result of 0
- So we fill the cell with the resulting value and save the direction this value was derived from as well (red arrow)

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0		
G	-12				
P	-18				
K	-24				
K	-30				
P	-36				

- Like this we can now fill the second line of our matrix one cell at a time and the result than looks as shown on the right
- Now each cell in the second row contains the scores and we know for each cell, which neighbor was used to calculate this score

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12				
P	-18				
K	-24				
K	-30				
P	-36				

- Now we move to the next line and fill it one by one in the same manner as we did for the previous line
- We start by calculating the scores for the first empty cell, the mismatch G/M with a PAM250 score of -3, according to our three possible ways:
 - ▶ coming from the left: $-12 + (-6) = -18$
 - ▶ coming from the top: $6 + (-6) = 0$
 - ▶ coming from diagonal: $-6 + -3 = -9$

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12				
P	-18				
K	-24				
K	-30				
P	-36				

- Since the highest score was 0, stemming from the neighbor at the top we save this score and the direction it was derived from
- Like this we are now able to fill our grid cell by cell

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12	0			
P	-18				
K	-24				
K	-30				
P	-36				

- Sometimes more than one path yields the highest score, in these case we have to save all of them
- For example the cell for the match KK (red, Pam250 Score = 5) in the sixth row gets the following scores:
 - ▶ coming from the left: $-7 + (-6) = -13$
 - ▶ coming from the top: $10 + (-6) = 4$
 - ▶ coming from diagonal: $-1 + 5 = 4$

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12	0	11	5	-1
P	-18	-6	5	10	11
K	-24	-12	-1	10	9
K	-30	-18	-7		
P	-36				

- In this case we save the calculated score for this cell (4) and both directions it could be calculated from (red arrows)

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12	0	11	5	-1
P	-18	-6	5	10	11
K	-24	-12	-1	10	9
K	-30	-18	-7	4	
P	-36				

- The finished grid than looks as shown on the right side
- We also now know all possible paths through the matrix, since any combinations of arrows that lead us from the top left corner of our grid (red 0) to the bottom right corner of our grid (green 10) are possible Alignments
- However the number of paths is still too large to run a simple comparison

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12	0	11	5	-1
P	-18	-6	5	10	11
K	-24	-12	-1	10	9
K	-30	-18	-7	4	9
P	-36	-24	-13	-2	10

- Instead of going forward (red to green), we now move backwards (green to red) through our grid by using a method called backtracking
- We start at the bottom right corner of our grid and follow the arrows in reverse
- So in the first step we move from the Match P/P to the match K/K since this is the only path available to us here

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12	0	11	5	-1
P	-18	-6	5	10	11
K	-24	-12	-1	10	9
K	-30	-18	-7	4	9
P	-36	-24	-13	-2	10

- As long as only one path (one arrow) is available to us we simply follow these arrows
- However once we encounter a cell that has more than 1 possible path leading to it we need to compare the scores of the corresponding starting points of these paths

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12	0	11	5	-1
P	-18	-6	5	10	11
K	-24	-12	-1	10	9
K	-30	-18	-7	4	9
P	-36	-24	-13	-2	10

- In our example we could go to the top (red 10) of the current cell (green 4) or we could go to the diagonal neighbor (red -1)
- In such cases we follow the path that leads to the higher score (here the 10)
- If multiple of these paths lead to the same score, both represent usable paths and we would generate more than just 1 solution

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12	0	11	5	-1
P	-18	-6	5	10	11
K	-24	-12	-1	10	9
K	-30	-18	-7	4	9
P	-36	-24	-13	-2	10

- Like this we now backtrack through our grid cell by cell until we reach the top left corner (red 0)

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12	0	11	5	-1
P	-18	-6	5	10	11
K	-24	-12	-1	10	9
K	-30	-18	-7	4	9
P	-36	-24	-13	-2	10

- The finished backtracking than looks as shown on the right side and the resulting Alignment would be:

M G _ K _ P

M G P K K P

- This would be the result of the Needleman-Wunsch Algorithm for these two sequences
- The overall score for this Alignment would be:
 - ▶ $10 = 6 + 5 + -6 + 5 + -6 + 6$

		M	G	K	P
	0	-6	-12	-18	-24
M	-6	6	0	-6	-12
G	-12	0	11	5	-1
P	-18	-6	5	10	11
K	-24	-12	-1	10	9
K	-30	-18	-7	4	9
P	-36	-24	-13	-2	10

- As we learned a global alignment delivers the best results if both sequences have nearly the identical length
- Therefore semi-global and local Alignments were developed to work with sequences that differ in their lengths
- Both use similar approaches to global Alignments and work with the same grid and backtracking mechanism we saw for the Needleman-Wunsch Algorithm
- However there are some differences in the grid creation and backtracking principles

- For semi-global alignments the cells of the first column and first row of the grid are filled with 0 instead of the calculated values we saw earlier
- The backtracking then starts at the highest score of the last row or last column of our grid (green 11)

		M	G	K	P
	0	0	0	0	0
M	0	6	0	-6	-12
G	0	0	11	5	-1
P	0	-6	5	10	11
K	0	-12	-1	10	9
K	0	-18	-7	4	9
P	0	-24	-13	-2	10

- The backtracking ends as soon as we reach either the first row or first column of our grid
- So the resulting semi-global Alignment for our two sequences would be:

M G K P

M G _ P K K P

		M	G	K	P
	0	0	0	0	0
M	0	6	0	-6	-12
G	0	0	11	5	-1
P	0	-6	5	10	11
K	0	-12	-1	10	9
K	0	-18	-7	4	9
P	0	-24	-13	-2	10

- For calculating local alignments Temple F. Smith and Michael S. Waterman proposed an algorithm in 1981
- It is a variation of the Needleman-Wunsch algorithm and thus follows also very similar principles
- During the grid creation the lowest possible score is 0, that means if a cell would get a negative value we fill it with a 0 instead
- Furthermore only paths that lead to scores bigger than 0 are saved during the grid creation

- On the right side you can see the finished grid of the Smith-Waterman algorithm
- As you can see the amount of possible paths was reduced significantly in comparison to the Needleman-Wunsch algorithm

		M	G	K	P
	0	0	0	0	0
M	0	6	0	0	0
G	0	0	11	5	0
P	0	0	5	10	11
K	0	0	0	10	9
K	0	0	0	4	9
P	0	0	0	0	10

- The traceback then starts at the cell that contains the biggest score of all cells in our grid (green 11) and it ends as soon as we hit a 0 (red 0)
- If we have multiple cells with the same maximal score, tracebacks are calculated from all of them, but only the traceback with the highest overall score is returned

		M	G	K	P
	0	0	0	0	0
M	0	6	0	0	0
G	0	0	11	5	0
P	0	0	5	10	11
K	0	0	0	10	9
K	0	0	0	4	9
P	0	0	0	0	10

Diagram illustrating the Smith-Waterman alignment grid and traceback path. The grid shows scores for alignments between sequences M, G, K, and P. The highest score (11) is highlighted in green, and the starting point for traceback (0) is highlighted in red. Arrows indicate the traceback path from the green cell back to the red cell.

- So the resulting local Alignment for our two sequences using the Smith-Waterman algorithm would be:

M G K P

M G _ P K K P

- In this case the semi-global and the local alignments are identical

		M	G	K	P
	0	0	0	0	0
M	0	6	0	0	0
G	0	0	11	5	0
P	0	0	5	10	11
K	0	0	0	10	9
K	0	0	0	4	9
P	0	0	0	0	10

Analyzing Alignments

- When analyzing Alignments keep in mind that you can only compare different Alignments for the same sequences with each other
- As soon as you have Alignments with different sequences you can't compare any numerical values anymore
- Furthermore you can't compare editing distances to scores

- Once an Alignment was calculated there are several numerical values that can help us in analyzing it:
 - ▶ Length → Number of columns in the Alignment, should be near the length of the longer sequence
 - ▶ Identity → Number of matches, should be as high as possible
 - ▶ Similarity → Number of mismatches of similar amino acids, should be as high as possible
 - ▶ Gaps → Number of letters aligned to nothing, should be as low as possible
 - ▶ Overall Score → Sum of all match, mismatch and gap costs over the whole Alignment, should be as high as possible

- Aside of these numerical values there are many other things you can look at when analyzing your pairwise Alignments:
 1. The amount of gaps
 2. The placement of gaps, inside or outside of specific domains
 3. Regions with a disproportionately large amount of matches
 4. Influence of Low-complexity regions
 5. Comparing the Alignment to a secondary structure prediction

Examples A0A5N3XQ52 / A0A9D3NLI7

Examples B4DTL8 / B3JI28

- During the Dotplot part you compared four sequences carrying a TRUD Domain (M3Z3I3.fasta, Q96PZ0.fasta, A0A5N3XQ52.fasta and A0A9D3NLI7.fasta)
- Use these sequences to run pairwise Alignments using EMBOSS Needle and EMBOSS Water to compare the sequence Q96PZ0.fasta to the other three (6 Alignments in total)
- Make sure to adjust the matrix to fit your results
 - ▶ Start with BLOSUM62 and look at the calculated identity
 - ▶ Afterwards restart the Alignment with a different matrix, BLOSUM90 for example
- How do the results differ between the two tools?
- Which sequence is the most similar one to Q96PZ0.fasta?

- Use the sequences XP_039961828.fasta and NP_477249.fasta to calculate a pairwise Alignment using EMBOSS Needle with the default parameters
- Calculate the same Alignment while using matrices BLOSUM40 and BLOSUM45
- Which of the three matrices you used produced the best Alignment?
- Afterwards use the matrix that produced the best result to calculate new Alignments but this time adjust the gap penalties as follows:
 - gap_open penalty = 5, gap_extend penalty = 0.5
 - gap_open penalty = 1, gap_extend penalty = 0.5
 - gap_open penalty = 1, gap_extend penalty = 0
- Which of the four Alignments do you think is the best one?

Pairwise Alignments in Python

- We can calculate pairwise Alignments with the help of Python as well
- The biopython module contains two submodules pairwise2 and Align that contain functions for the calculation of these alignments
- When working with these submodules we always need the sequences to be in the form of strings
- Alternatively we can use biopython sequence objects generated with the help of the SeqIO parser that is also part of the biopython module

- The best way of calculating Alignments in python is by using the submodule Align of biopython
- We can access this submodule with the following import:

```
from Bio import Align
```

- The main advantage of the submodule Align is that it offers much more adjustments for our calculations than the relative simple pairwise2 submodule
- Furthermore both use the same algorithms in the back and thus there is no difference between them
- Last but not least pairwise2 is on the removal list for biopython, so it could be removed from the module in the near future

- When using this submodule we always need to create a PairwiseAligner object first:

```
aligner = Align.PairwiseAligner()
```

- Once we are finished setting up the different parameters we can calculate our alignments as follows:

```
alignments = aligner.align(seq1.seq, seq2.seq)
```

- Aside of calculating the whole Alignment we are also able to just calculate the score of our alignment by using:

```
score = aligner.score(seq1.seq, seq2.seq)
```

- The PairwiseAligner can return multiple Alignments at once, depending on your settings and the sequences in question
- All these Alignments produced the same overall score
- To see how many alignments you got back simply use the built-in method `len()` on the PairwiseAlignments object `alignments` created earlier:

```
print(len(alignments))
```


- If not specified differently the Pairwise Aligner uses the following default parameters:
 - ▶ match score of 1
 - ▶ mismatch score of 0
 - ▶ linear gap cost of 0
- So you need to specify which kinds of costs you want the Aligner to use

- As we learned in most cases we need to use specific scoring methods and gap costs to model the biological reality when calculating our alignments
- One of the first things that should come to your mind is to use a substitution matrix instead of a general match/mismatch score
- Biopython contains a class called `substitution_matrices` which is part of the `Align` submodule
- This class allows us to import different substitution matrices into our python script

- To load a specific matrix you need to use the `load()` method of the `substitution_matrices` class of `Align`:

```
blosum62 = Align.substitution_matrices.load("BLOSUM62")
```

- The following matrices are available to us:
 - ▶ 'BENNER22', 'BENNER6', 'BENNER74', 'BLOSUM45', 'BLOSUM50', 'BLOSUM62', 'BLOSUM80', 'BLOSUM90', 'DAYHOFF', 'FENG', 'GENETIC', 'GONNET1992', 'HOXD70', 'JOHNSON', 'JONES', 'LEVIN', 'MCLACHLAN', 'MDM78', 'NUC.4.4', 'PAM250', 'PAM30', 'PAM70', 'RAO', 'RISLER', 'SCHNEIDER', 'STR', 'TRANS'

- Once you loaded the matrix you can hand it over to the Pairwise Aligner with the attribute `substitution_matrix`:

```
aligner.substitution_matrix = blosum62
```

- Now the Pairwise Aligner will use the specified matrix for all alignments you want to calculate

- The next step is to assign different gap penalties
- The Pairwise Aligner has many different types of gap penalties (later slide) that allow you to adjust everything to your needs
- Here we will focus on the normal affine gap costs, by using the attribute `open_gap_score` and `extend_gap_score`:

```
aligner.open_gap_score = -10
```

```
aligner.extend_gap_score = -0.5
```

- These scores apply to all gaps introduced into the Alignment, even gaps at the start or end of the Alignment

- Per default the alignment that is calculated by using the Pairwise Aligner object is a global alignment
- If you want to calculate a local alignment you need to adjust the alignment mode by adding the following code to your script:

```
aligner.mode = "local"
```

- Earlier I mentioned that the Pairwise Aligner contains a lot of different gap penalties you can adjust
- Two of the common ones are the `end_open_gap_score` and `end_extend_gap_score` which tell the Aligner how to deal with gaps at the end / beginning of the alignment:

```
aligner.end_open_gap_score = 0
```

```
aligner.end_extend_gap_score = 0
```

- By setting these penalties to 0 you can try to force the algorithm into calculating a semi-global alignment

- Instead of using affine gap costs you can switch to linear gap costs as well
- To do so you can use only the `extend_gap_cost` attribute of the Aligner:
 - `aligner.extend_gap_cost = -6`
- Like this you can make use of linear gap costs, since all gaps are punished in the same manner

- When working with the Pairwise Aligner you need to keep in mind that it always uses the parameters you specified last
- That means you can overwrite parameters:

```
aligner.gap_score = -6
```

```
aligner.open_gap_score = -10
```

```
aligner.extend_gap_score = -0.5
```

- In the example above the Aligner would now use the affine gap costs I specified last when calculating the next Alignment

- Aside of the examples you saw so far, regarding gap costs, the Pairwise Aligner contains many more parameters
- In general you can adjust the gap parameters for each sequence separately with the first sequence being called target and the second sequence being called query, e.g. `target_open_gap_score`
- You can also adjust the parameters for gaps at the beginning, called left, or the end, called right, of the alignment, e.g. `left_open_gap_score`

- The calculated Alignment objects have several different methods and attributes you can make use of
- The first one is the method `counts()` that shows you the amount of matches (identities), gaps and mismatches:

```
alignments[0].counts()
```

- The created object allows you to access the numbers in it by using different attributes:

```
alignments[0].counts().gaps
```

```
alignments[0].counts().identities
```

```
alignments[0].counts().mismatches
```

- Two additional attributes that can be useful are:
 - ▶ `frequencies` that returns the frequency of each letter in each column of the alignment
 - ▶ `substitutions` that returns an `Array` with the number of substitutions of letters in the alignment

- Use the sequences B3JI28.fasta and B4DTL8.fasta to calculate pairwise Alignments with the help of the ALIGN submodule of Biopython
- Use the matrices PAM250, BLOSUM45, MDM78, JOHNSON, BENNER6 and GONNET1992
- Calculate local and global Alignments for all six matrices
- What are the differences?
- Which combination produced the best result in your opinion?

- You compared four sequences carrying a TRUD Domain during the earlier parts
- Use these sequences to run local and global pairwise Alignments in Python using the ALIGN submodule of biopython to compare the sequence Q96PZ0.fasta to the other three
- Make sure to adjust the matrix to fit your results while starting with BLOSUM62
- Use the method counts() of your alignment objects (alignments[0].counts()) to see the number of matches, mismatches and gaps for the alignment
- Afterwards restart the Alignment with a different matrix, BLOSUM90 for example
- How do the results differ between the two versions?
- Which sequence is the most similar one to Q96PZ0.fasta?

- Read in the fasta file Sequences_MSA_01.fasta
- Calculate the scores for all possible pairwise Alignments for the 100 sequences in it
- Save the results in a pandas DataFrame

Multiple Sequence Alignments

- You saw how you can compare two sequences with each other using pairwise Alignments as calculated with the help of Needleman-Wunsch or Smith-Waterman
- In most cases however we have to compare many different sequences (more than 100) with each other at the same time
- In these cases we need Multiple Sequence Alignments, MSAs in short, to be able to do this

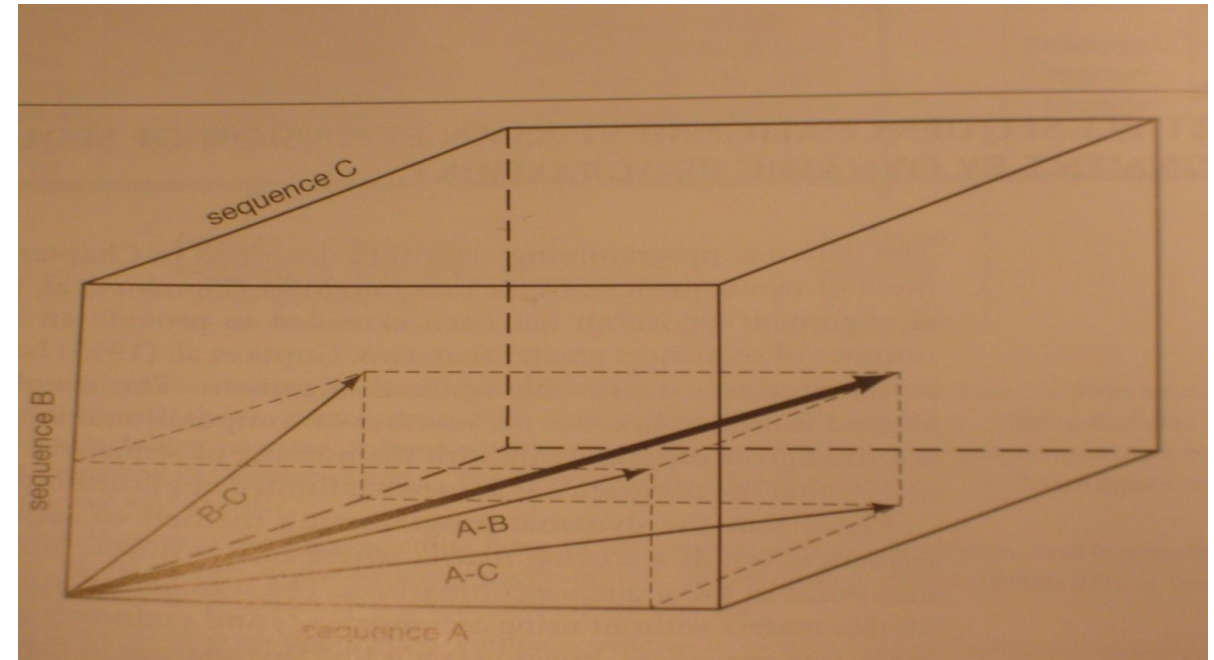
- Phylogenetic research is always based on MSAs since they allow us to visualize the similarities between different sequences
- Furthermore are MSAs the basis for phylogenetic trees which show the relationship between organisms

- When calculating MSAs we have to redefine how to score our Alignments since we are now comparing multiple pairs of letters and not just one pair
 - E.g. costs over all columns are added up for all possible pairs
- There are exact and heuristic approaches for calculating such scores
- Most programs use heuristic approaches, simply because exact approaches take much too long

- Is the Art, to get to probable statements or practicable solutions with limited knowledge (incomplete information) and limited time
- Get the best possible solution with the least amount of work
- Examples that employ heuristics include using trial and error, a rule of thumb or an educated guess
- We use heuristic methods to get to solutions of very complex and time consuming processes with a short calculation time and small amount of steps

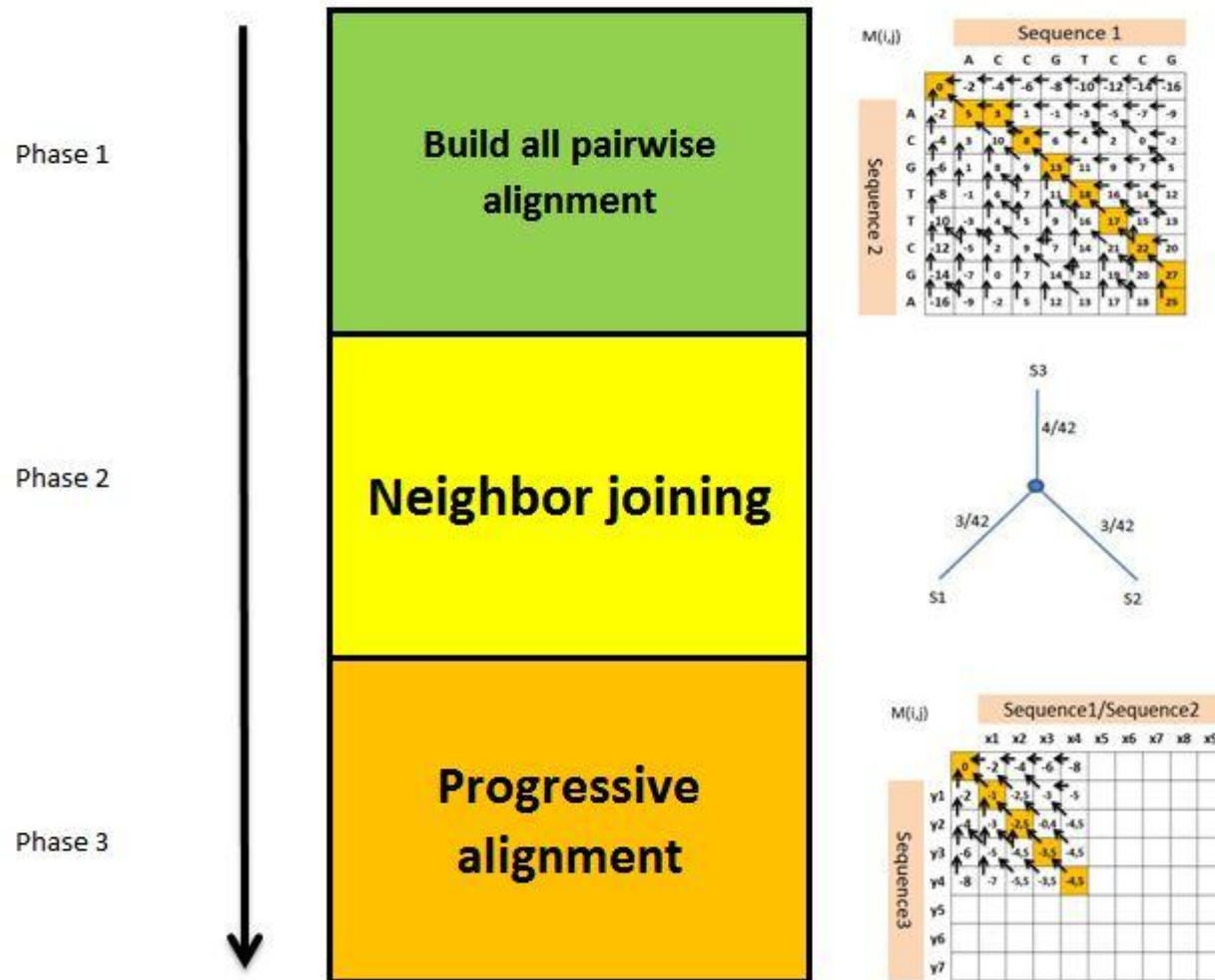
- In informatical algorithms heuristics are used to reach the best possible solutions with the smallest amount of calculation time
- They are mostly used when a solution can't be calculated at all or would take too much time to be calculated
- By using Shortcuts or approximations heuristic methods allow us to do complex tasks in a relative fast and good manner
- Heuristic algorithms do NOT produce optimal results but results that are as near as possible to the optimal result

- An algorithm called “3D”-Needleman-Wunsch was proposed to calculate optimal MSAs
- Instead of aligning two sequences, N sequences are considered
- With each sequence that has to be aligned additionally, the number of possibilities increases exponentially
- Too CPU-intensive to be used for more than 3 sequences



- Instead of using an algorithm that calculates an optimal MSA for our sequences we use algorithms that calculate so called progressive or hierarchical alignments
- The Idea behind these alignments is to find the best pairwise alignments and build the MSA around them
- This is done with the help of a guide-tree that tells us the order in which our sequences need to be aligned to each other
- The results for these algorithms are often “suboptimal” but good enough to detect similarities between the sequences
- One of the first and still used software that handles this is ClustalW

The three phases of the CLUSTALW algorithm



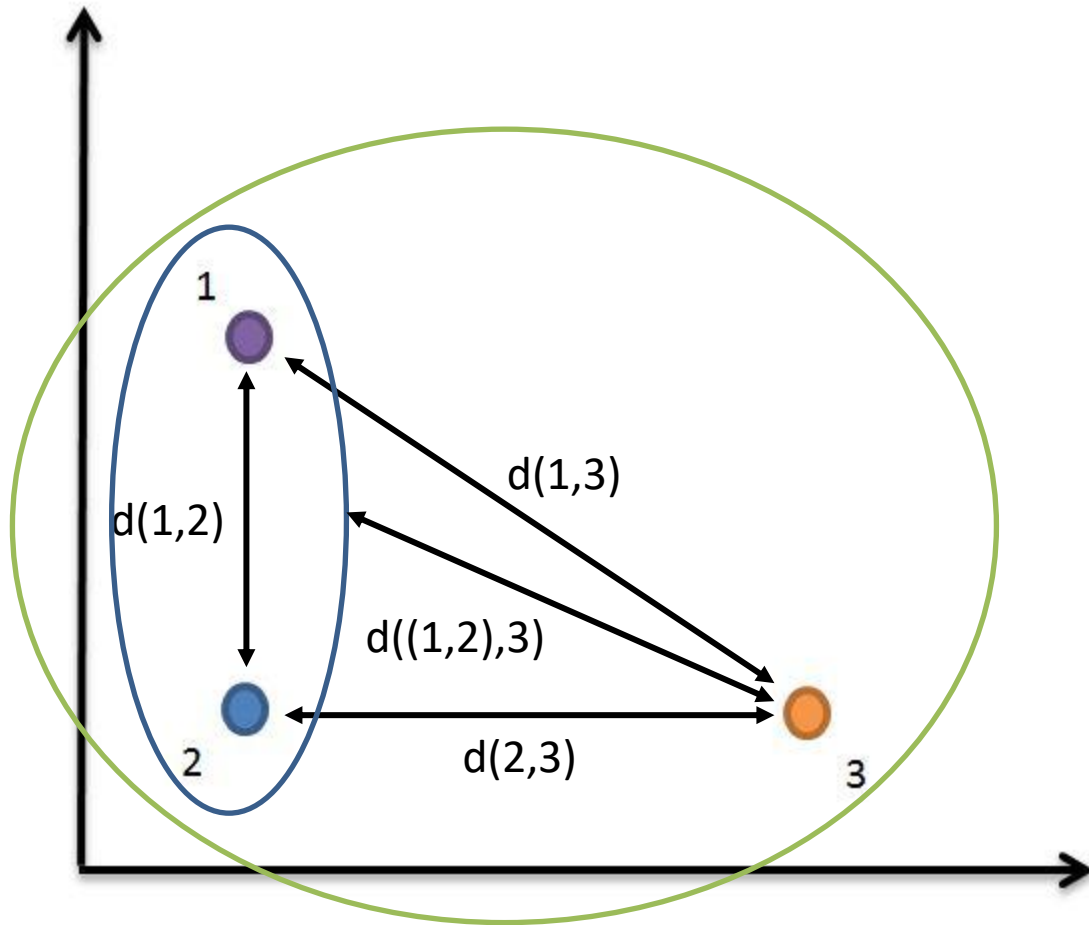
- ClustalW consists of three phases:
 1. We calculate and score all pairwise Alignments for all sequences
 2. We create a guide tree using a neighbor joining approach
 3. Calculate the MSA by using an At-Alignment approach

- For each pair of sequences optimal alignments are generated via Needleman Wunsch
- The editing distance for each pair is used as a measure of distance between the sequences
- All editing distances are collected in a distance matrix which has the sequences as columns and rows and each field contains the corresponding editing distance
- This matrix is then used in the second step to calculate the neighbor-joining tree

	Spinach	Rice	Mosquito	Monkey	Human
Spinach	0.0				
Rice	84.9	0.0			
Mosquito	105.6	117.8	0.0		
Monkey	90.8	122.4	84.7	0.0	
Human	86.3	122.6	80.8	3.3	0.0

- The result of the first phase of ClustalW is a table similar to the one shown above
- The values in the different cells of table represent the editing distances calculated from optimal global alignments between the two sequences in question

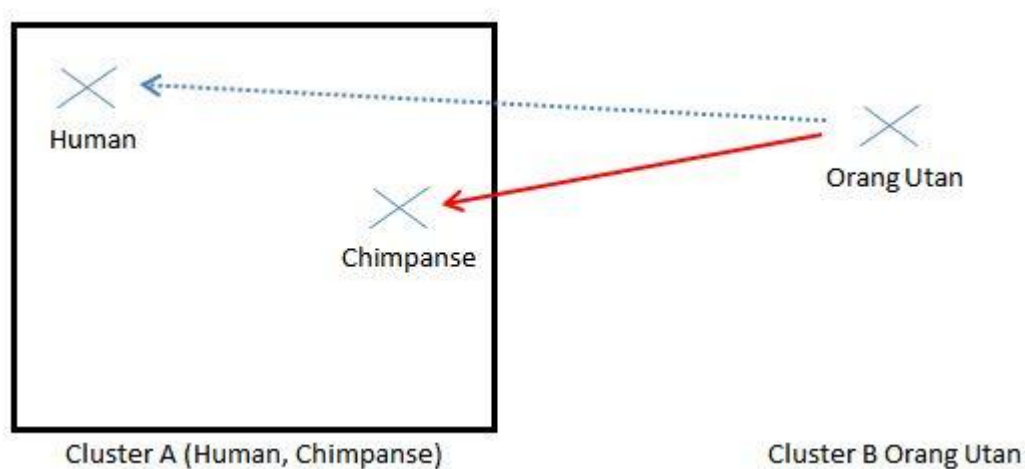
- In the second phase of the ClustalW algorithm we need to calculate the guide tree with the help of a cluster method
- In a first step of this clustering process the two sequences that have the smallest editing distance are taken as first cluster for our Tree
 - ➔ IDEA: Similar sequences divided late from a common ancestor and thus have to be relatively low in the phylogenetic tree indicated by a low editing distance
- Afterwards new distances for the remaining sequences against this first pair / cluster have to be calculated with the help of so called update functions



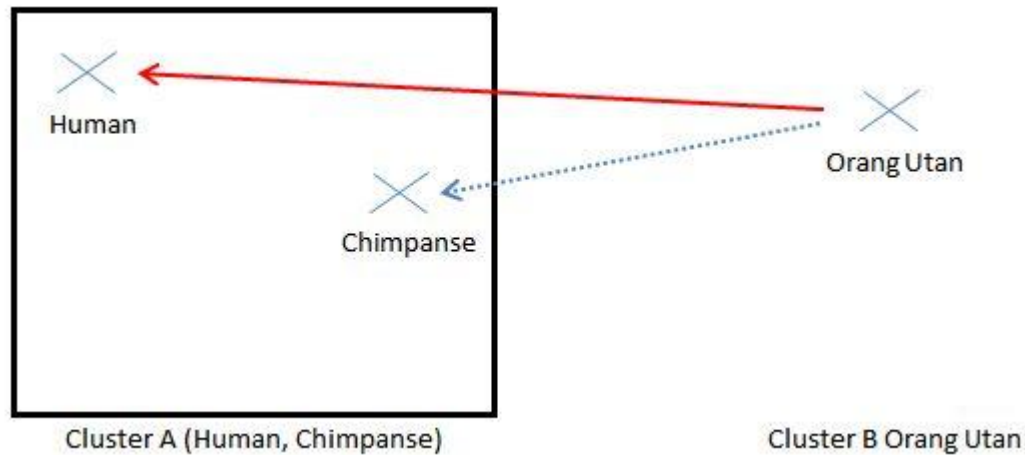
- Imagine we have these 3 Sequences
- Now we calculate the distances between all three of them
- Since the distance $d(1,2)$ between 1 and 2 was the smallest we combine them to a cluster
- Calculate the new distance from Cluster $\frac{1}{2}$ to node 3
- Combine all nodes to a big Cluster

- We saw that an essential part of clustering is the calculation of new distances between our clusters and the remaining sequences
- There are many different Update functions available and they define the method
- The three easiest ones are:
 - Single linkage
 - Complete linkage
 - UPGMA

- The new distance is the shortest distance of all available distances between our remaining sequences and the already clustered sequences



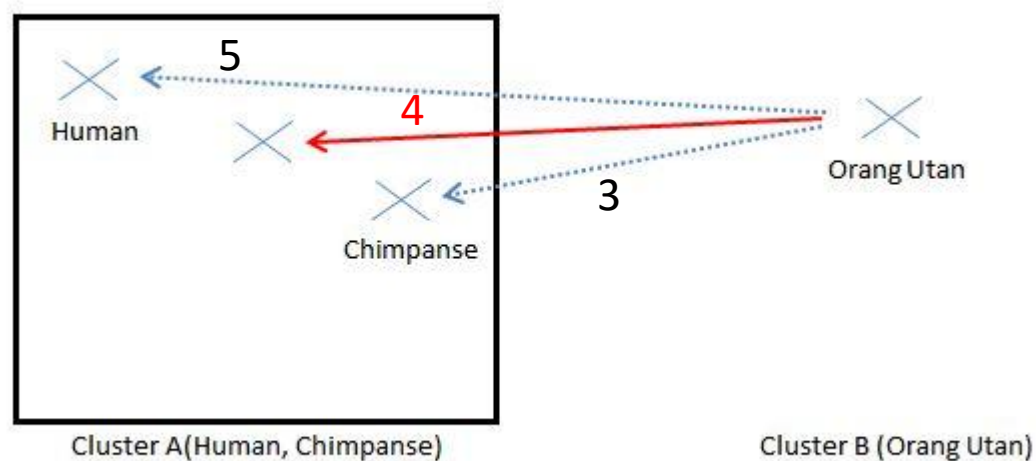
- The new distance is the longest distance of all available distances between our remaining sequences and the already clustered sequences



- UPGMA stands for unweighted pair group method with arithmetic mean

$$D(neu, alt) = \frac{D(neu_links, alt) + D(neu_rechts, alt)}{2}$$

$$D(HuChi, Orang) = \frac{D(Human, Orang) + D(Chimpanse, Orang)}{2}$$



$$D(HuChi, Orang) = \frac{5 + 3}{2} = 4$$

	Spinach	Rice	Mosquito	Monkey	Human
Spinach	0.0				
Rice	84.9	0.0			
Mosquito	105.6	117.8	0.0		
Monkey	90.8	122.4	84.7	0.0	
Human	86.3	122.6	80.8	3.3	0.0

- Now we can visualize the clustering process and tree generation with the help of the distances we saw earlier
- In a first step we need to determine the smallest editing distance between all sequence pairs (red circle)

	Spinach	Rice	Mosquito	Monkey	Human
Spinach	0.0				
Rice	84.9	0.0			
Mosquito	105.6	117.8	0.0		
Monkey	90.8	122.4	84.7	0.0	
Human	86.3	122.6	80.8	3.3	0.0

- Since Monkey and Human have the smallest distance we can create our first cluster as follows:



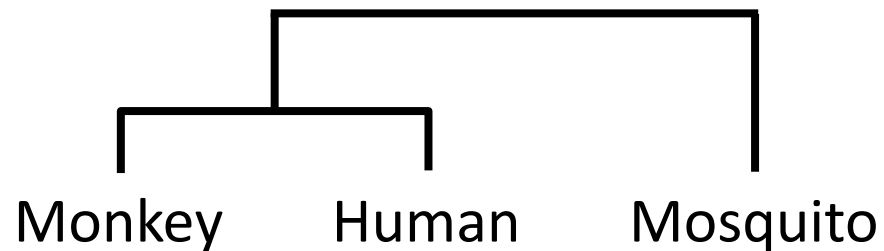
	Spinach	Rice	Mosquito	Mon/Human
Spinach	0.0			
Rice	84.9	0.0		
Mosquito	105.6	117.8	0.0	
Mon/Human	88.6	122.5	82.8	0.0

- In the next step we calculate the distances of the remaining three sequences to the Monkey/Human cluster (red circles)
- Afterwards we again search for the smallest distance to determine which sequence needs to be added next (green circle)



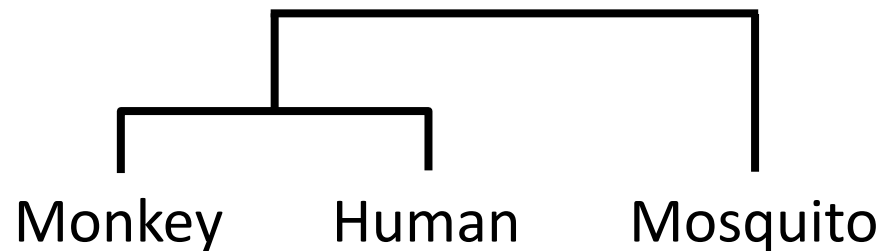
	Spinach	Rice	Mosquito	Mon/Human
Spinach	0.0			
Rice	84.9	0.0		
Mosquito	105.6	117.8	0.0	
Mon/Human	88.6	122.5	82.8	0.0

- Now we can add Mosquito to our tree



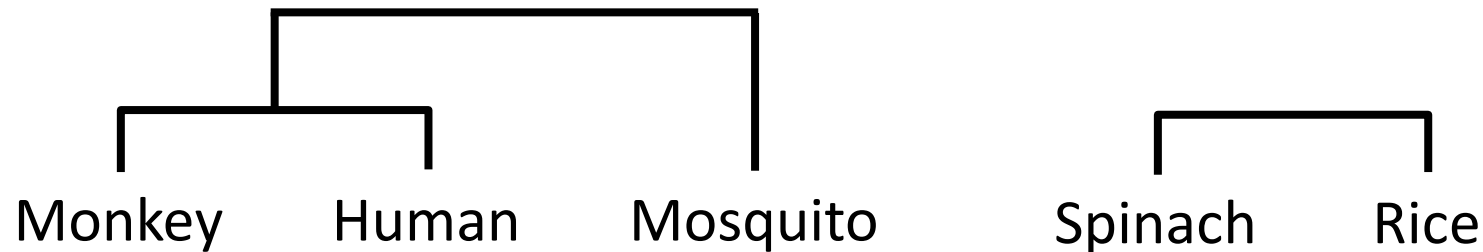
	Spinach	Rice	(Mon/Human)/Mos
Spinach	0.0		
Rice	84.9	0.0	
(Mon/Human)/Mos	97.1	120.2	0.0

- In the next step we again calculate the new distances (red circles)
- Afterwards we determine again the smallest available distance (green circle)



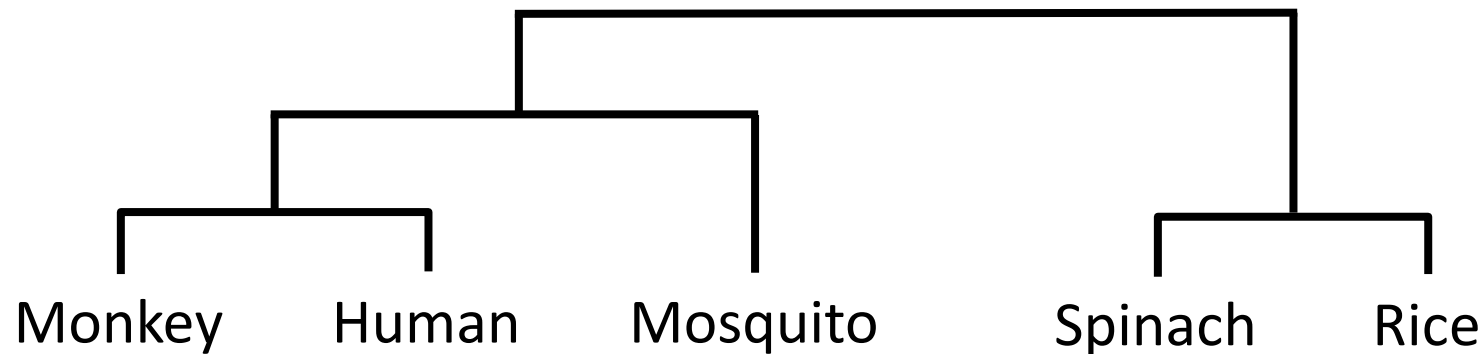
	Spinach	Rice	(Mon/Human)/Mos
Spinach	0.0		
Rice	84.9	0.0	
(Mon/Human)/Mos	97.1	120.2	0.0

- Now we need to create a new cluster aside of the already existing one



	Spinach/Rice	(Mon/Human)/Mos
Spinach/Rice	0.0	
(Mon/Human)/Mos	108.7	0.0

- In the last step we calculate again the new distances but since we only have two clusters left we can simply combine them



- In the original algorithm the clustering method of “Neighbor Joining” is nowadays used when creating the guide tree
- This method follows the same principle but uses more complex calculation methods to determine the new distances between the sequences and the clusters
- However UPGMA was used in the very first version of ClustalW

- First the most similar sequences (the lowest cluster) are exactly, globally aligned
- Than other sequences are “at-aligned” to this global pairwise alignment
 - An alignment can be aligned with a sequence or another complete alignment
- During this process the original alignment can NOT be changed with the exception of introducing gaps in both sequences at the same time
 - That means the Matches/Mismatches are kept intact over the whole process

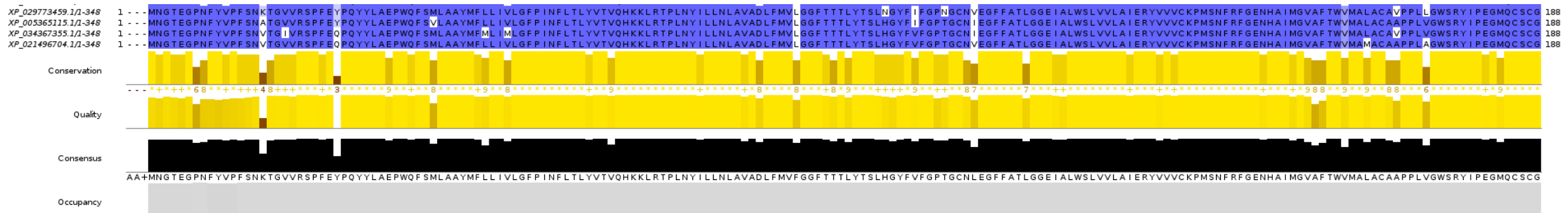
- We have many tools to analyze MSAs
- A simple method is to look at the MSA with the help of programs like Jalview which can give you an overview of conserved regions and regions with small similarity

Beratung
 +Bildung

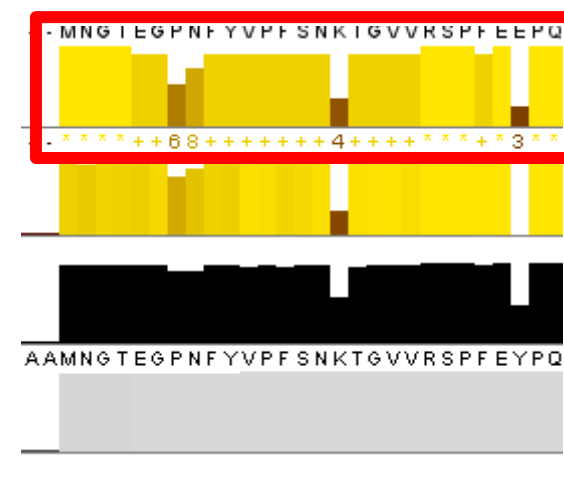
[illegible]



- The two images you saw before showed you different ways of looking at MSAs using Jalview
- While the Alignment overview with all sequences might be useful, we are more interested in the bar diagrams at the bottom of the window
- These contain some quality measurements to analyze your MSA even further

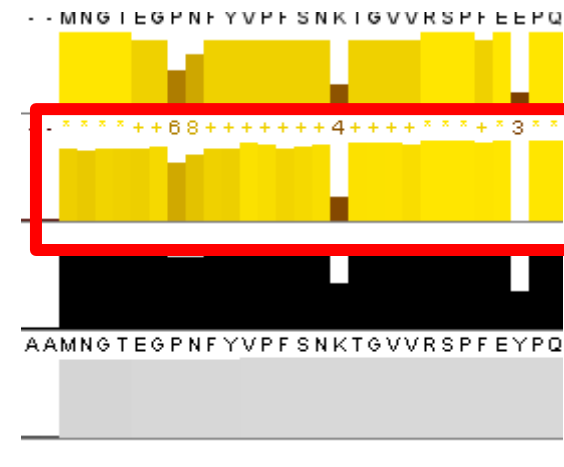


- The first diagram displays a measurement for the conservation of the sequences (red rectangle)
- The value range from 0 (no conservation) up to 11 (total conservation)
- Since the space is limited JalView prints a + for a conservation of 10 and a * for a conservation of 11
- The 11 represents total conservation that means we have only one amino acid in the whole column
- A 10 represents columns with mutations where all properties are conserved

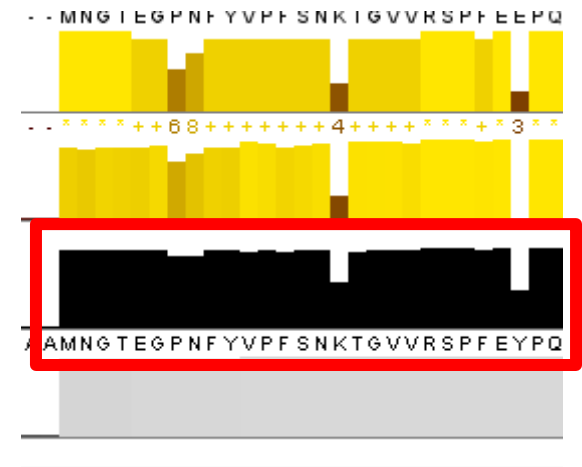


- The conservation is a calculated quantitative alignment annotation
- It measures the number of conserved physico-chemical properties conserved for each column of the alignment
- Its calculation is based on the one used in the AMAS method of multiple sequence alignment analysis [19]

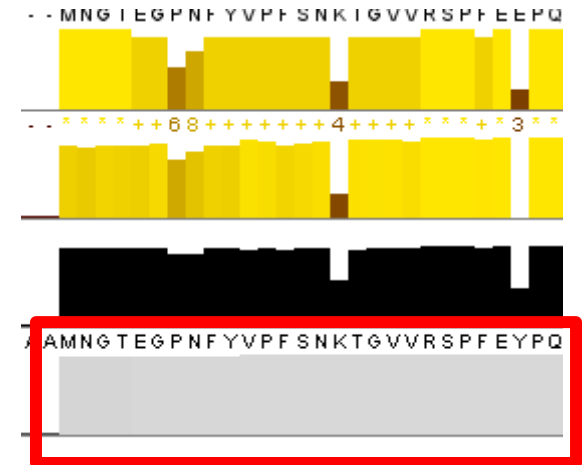
- The second diagram displays a measurement for the quality of the alignment in each column (red rectangle)
- The quality score is calculated for each column in an alignment
- This is done “by summing, for all mutations, the ratio of the two BLOSUM62 scores for a mutation pair and each residue's conserved BLOSUM62 score (which is higher)”
- This value is normalized for each column



- The third diagram displays the consensus of the alignment in each column (red rectangle)
- The consensus of a column in a MSA refers to the amino acid or nucleotide that appears the most in the corresponding column
- The letters represent this consensus and the bars show the percentage value for the appearance in the column
- A value of 100% means the consensus letter appears in each column
- If more than letter could be the consensus a + is displayed



- The fourth diagram represents the occupancy of each column in the MSA (red rectangle)
- This refers to amount of letters in each column
- For all diagrams the genral rule is, the bigger the bar the better is the value



Example Sequences_MSA_01

- Use Clustal Omega and Clustal W to create MSAs for the file A0A5_fasta, A0A9_fasta and M3Z3_fasta
- What do you think about the quality of these MSAs?
- Why do you think the results are as they are?

- Use the two online tools below and run six MSAs on each of them, one for each of the big fasta files prepared for you
- Look at the results in Jalview and compare the results, that means compare the Clustal Omega result for Sequences_MSA_01 with the Clustal W result for the same fasta file
- <https://www.genome.jp/tools-bin/clustalw>
- <https://www.ebi.ac.uk/jdispatcher/msa/clustalo>

- A multitude of different MSA programs exist in the web, because in the first decade of the 2000s more than 50 programs were published
- Some of them are still online while some others were taken down or are not supported anymore over the course of time
- Five of the most prominent ones, which are all available at the EMBL or the NCBI, are T-COFFEE, MUSCLE, MAFFT, COBALT and K-ALIGN
- The main differences between these programs lie in the first step of calculating the distance matrix or the second step of creating the guide tree
- I will give you just a short outline of these programs
- If you want more information about them, I uploaded the original publications of these programs to the NextCloud

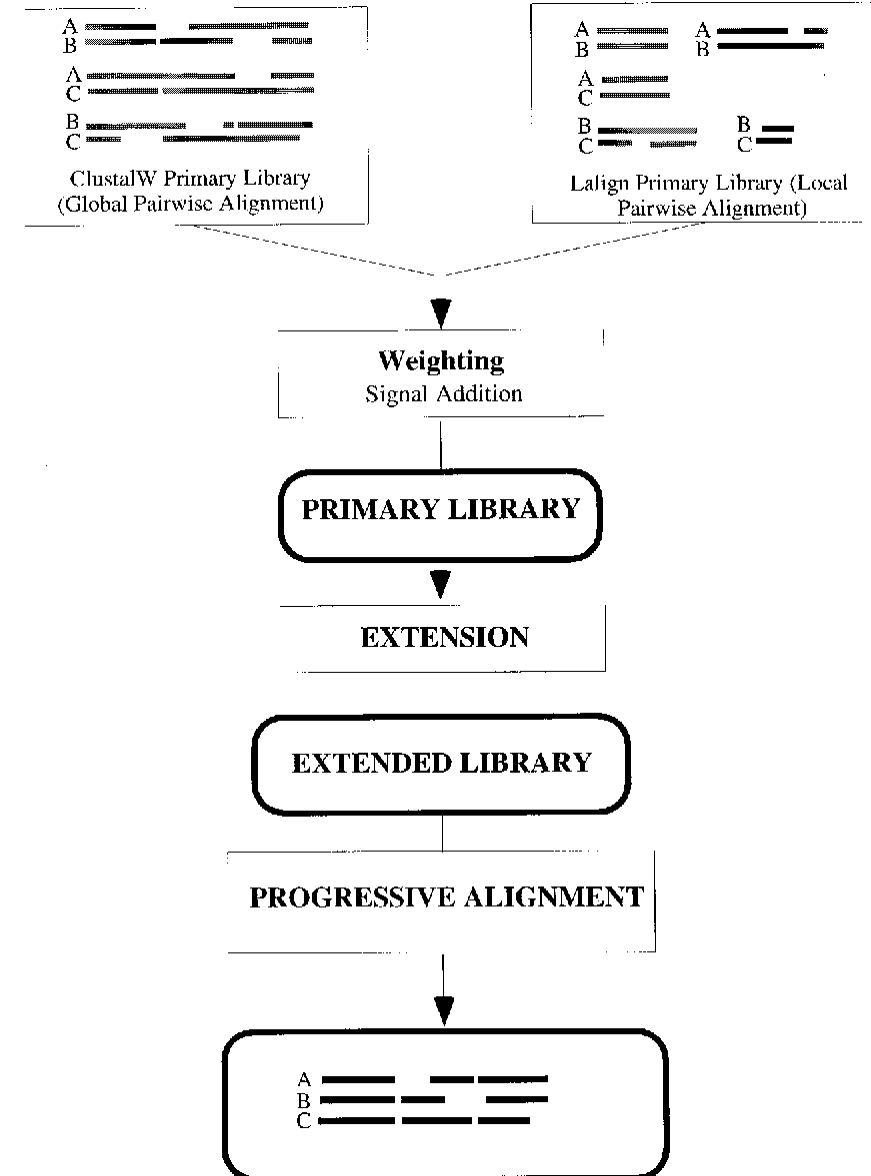
- K-ALIGN follows the same general strategy as ClustalW however it uses the Wu-Manber string-matching algorithm [21] for calculating the distance matrix and optionally when at-aligning the sequences
- The Wu-Manber algorithm is an extension to the exact Baeza-Yates-Gonnet algorithm [22] and allows string - matching with mismatches
- During the creation of the pairwise Alignments K-ALIGN uses tuples of three letters instead of using a Needleman-Wunsch approach
- These tuples are matched with while allowing one error in the matching
 - That means “AAA” could be aligned to “AAL”

- To avoid using random hits of these tuples in the Alignment each matched tuple is assigned an additional score of:
 - ▶ 16 for total matches between the two sequences
 - ▶ 8 if one sequence has a mismatch
 - ▶ 1 if both sequences have a mismatch
- These matches are collected in the Alignment matrix in form of diagonals
- The similarity score of K-ALIGN is then defined as sum of the three highest scoring diagonals found
- That means K-ALIGN doesn't calculate optimal Alignments in the first step and instead searches for regions with the highest identities

	A	L	C	A	L	C	A	L	C
A									
L									
C									
A									
T									
C									
A									
I									
H									

- The result of this first step than looks as shown on the right side for each pair of sequences
- The pairwise score is then the sum of the scores for the diagonals in green, blue and red
- Afterwards the guide tree is calculated using the UPGMA method

- Tree-based Consistency Objective Function for alignment Evaluation
- Combines signals from heterogeneous sources:
 - sequence-alignment programs
 - structure alignments
 - Threading
 - manual alignment
 - Motifs and specific constraints

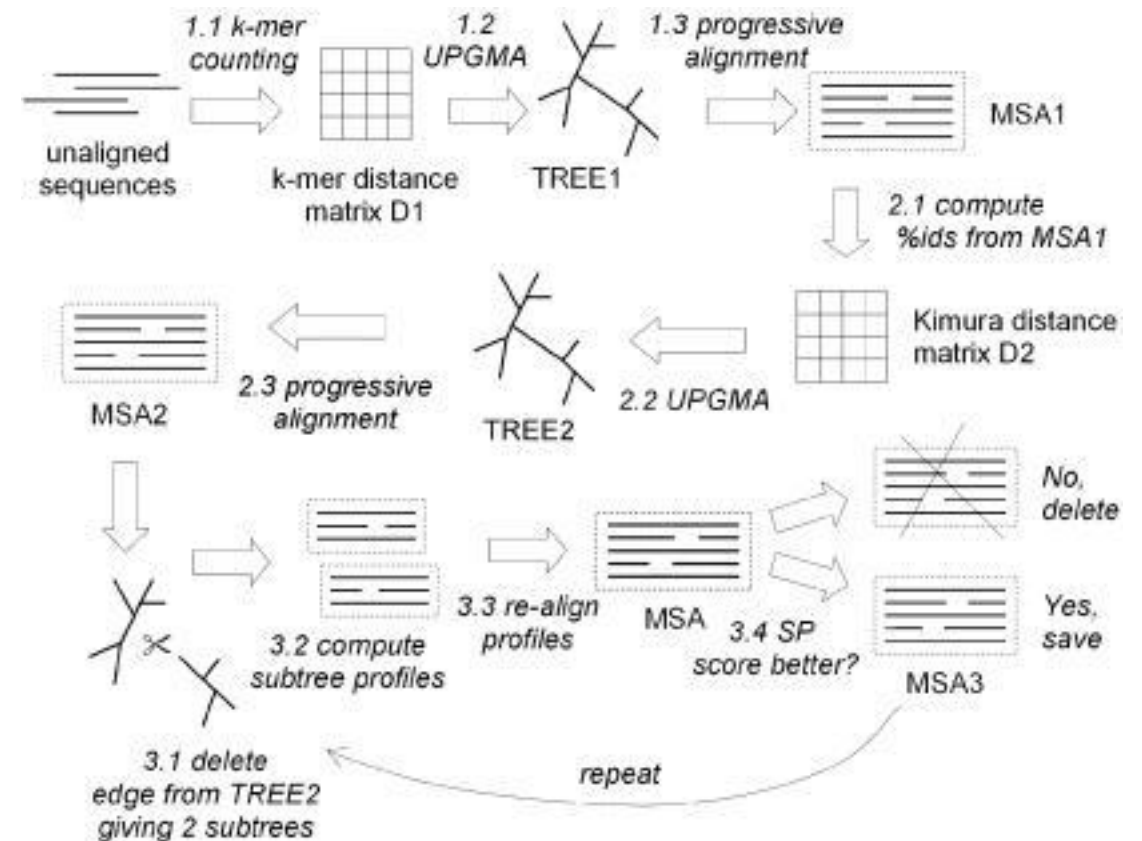


- For each pair of sequences a global Alignment is calculated using ClustalW and the top ten local Alignments are calculated using Lalign [24]
- Each Alignment is then represented as a list of pairwise residue matches and weighted according to the sequence identity
- After these two libraries were calculated they are merged together to form the primary library
- This primary library is then extended by adjusting the weights assigned to each sequence pair with the help of the remaining sequence pairs in the library

- Once the library was generated, the progressive Alignment process can be started
- First a guide-tree is calculated using the neighbor-joining method
- Using this guide-tree as basis the MSA than can be calculated in the same way as we learned for ClustalW

- Based on the fast Fourier transform (FFT)
- An amino acid a is assigned to a vector whose components are the volume value $v(a)$ and the polarity value $p(a)$ introduced by Grantham [4]
- Each sequence is converted into a sequence of normalized forms of these vectors
- Now the correlation between the two sequences can be calculated as a correlation between sequences of vectors with the help of Fourier transformations
- All in all this program is significantly faster than others without sacrificing the accuracy

- MUSCLE uses an iterative approach by calculating multiple distance matrices as well as multiple MSAs from these matrices
- The distance matrices are clustered using UPGMA to generate the guide-tree for the progressive Alignments



- To summarize MUSCLE in a simple way:
 1. We split up the sequences into smaller parts of 3 to 6 letters and compare how often they appear in a pair of sequences
 2. These values are then used as basis distance matrix for the first MSA
 3. Afterwards sequence identities are calculated for all sequence pairs using the first MSA as basis
 4. These identities are converted using a Kimura distance [28] to calculate a new distance matrix
 5. This distance matrix is then the basis for a second MSA
 6. From there small iterative adjustments are made to reach the best possible MSA

- The program COBALT tries to create MSAs of higher quality by:
 - ▶ Using biologically relevant information encoded in databases such as the conserved domain database (CDD) and PROSITE
 - ▶ Using local pairwise similarity present in *multiple* sequence pairs to highlight similar regions in otherwise divergent sequences
 - ▶ Specifying regions the users want to see aligned in any multiple alignment computed

- The general algorithm of COBALT consists of 6 steps:
 - ▶ Step 1: Find alignments for generating constraints.
 - ▶ Step 2: Find partial profiles and a pairwise consistent set of constraints.
 - ▶ Step 3: Generate a guide tree.
 - ▶ Step 4: Create a multiple alignment using the current set of constraints and guide tree.
 - ▶ Step 5: Create bipartitions and realign.
 - ▶ Step 6: Perform (optional) refinement by determining a new set of constraints and iterating from Step 4 as long as the number of constraints keeps increasing.

- K-ALIGN: <https://www.ebi.ac.uk/jdispatcher/msa/kalign>
- MAFFT: <https://www.ebi.ac.uk/jdispatcher/msa/mafft>
- MUSCLE: <https://www.ebi.ac.uk/jdispatcher/msa/muscle>
- T-COFFEE: <https://www.ebi.ac.uk/jdispatcher/msa/tcoffee>
- COBALT: https://www.ncbi.nlm.nih.gov/tools/cobalt/re_cobalt.cgi

Examples Sequences_MSA_01

- Use the file Rhodopsin.fasta to create MSAs with the help of the seven tools that were discussed during the course (ClustalW, Clustal Omega, MAFFT, K-Align, T-COFFEE, MUSCLE and COBALT)
- Compare the results and focus especially on the two big gaps at positions 240-270 / 400-420
- What other big differences (additional gaps over most parts of the MSA, alignments in less conserved regions (300-310 & 340-360)) can you spot?

- Use the file RasGTPase.fasta to create MSAs with the help of the seven tools that were discussed during the course (ClustalW, Clustal Omega, MAFFT, K-Align, T-COFFEE, MUSCLE and COBALT)
- Be aware that MUSCLE will take a relative long time
- Look at the resulting Alignments with the help of Jalview
- Do you think you could use these results in any further analysis?
- Create at least two Alignments using the file RasGTPase_Short.fasta afterwards and look again at the results
- Are these smaller MSAs better?

1. https://en.wikipedia.org/wiki/Sequence_homology (Last edited: 28.10.2022, Last visted:11.05.2023)
2. https://en.wikipedia.org/wiki/Conservative_replacement (Last edited: 17.12.2021, Last visted: 11.05.2023)
3. <https://judgestarling.tumblr.com/post/125812920996/radical-and-conservative-amino-acid> (Last edited: 03.08.2015, Last visited: 11.05.2023)
4. Grantham, R. "Amino acid difference formula to help explain protein evolution." Science (New York, N.Y.) vol. 185,4154 (1974): 862-4. doi:10.1126/science.185.4154.862
5. Miyata, T et al. "Two types of amino acid substitutions in protein evolution." Journal of molecular evolution vol. 12,3 (1979): 219-36. doi:10.1007/BF01732340
6. https://en.wikipedia.org/wiki/Point_accepted_mutation (Last edited: 30.09.2022, Last visited: 18.01.2023)
7. Dayhoff, M., R. Schwartz, and B. Orcutt. "22 a model of evolutionary change in proteins." Atlas of protein sequence and structure 5 (1978): 345-352.
8. Pearson, William. (2001). Protein sequence comparison and Protein evolution Tutorial - ISMB2000.
9. <https://en.wikipedia.org/wiki/BLOSUM> (Last edited: 04.01.2023, Last visited: 04.01.2023)

10. Henikoff, S, and J G Henikoff. "Amino acid substitution matrices from protein blocks." Proceedings of the National Academy of Sciences of the United States of America vol. 89,22 (1992): 10915-9.
doi:10.1073/pnas.89.22.10915
11. Cartwright, R.A. Logarithmic gap costs decrease alignment accuracy. *BMC Bioinformatics* 7, 527 (2006).
<https://doi.org/10.1186/1471-2105-7-527>
12. Needleman, S B, and C D Wunsch. "A general method applicable to the search for similarities in the amino acid sequence of two proteins." Journal of molecular biology vol. 48,3 (1970): 443-53. doi:10.1016/0022-2836(70)90057-4
13. https://en.wikipedia.org/wiki/Needleman%E2%80%93Wunsch_algorithm (Last edited: 05.04.2023, Last visited: 12.05.2023)
14. https://en.wikipedia.org/wiki/Smith%E2%80%93Waterman_algorithm (Last edited: 19.02.2023, Last visited: 15.05.2023)
15. <https://biopython.org/docs/1.75/api/Bio.pairwise2.html> (Last visited: 16.05.2023)

16. <http://biopython.org/DIST/docs/tutorial/Tutorial.html> (Last edited: 12.02.2023, Last visted: 16.05.2023)
17. <https://en.wikipedia.org/wiki/Heuristic> (Last edited: 10.01.2023, Last visited: 18.01.2023)
18. <https://de.wikipedia.org/wiki/Heuristik> (Last edited: 15.11.2022, Last visited: 18.01.2023)
19. Livingstone, C D, and G J Barton. "Protein sequence alignments: a strategy for the hierarchical analysis of residue conservation." Computer applications in the biosciences : CABIOS vol. 9,6 (1993): 745-56.
doi:10.1093/bioinformatics/9.6.745
20. Lassmann, T., Sonnhammer, E.L. Kalign – an accurate and fast multiple sequence alignment algorithm. BMC Bioinformatics 6, 298 (2005).
21. Sun Wu and Udi Manber. 1992. Fast text searching: allowing errors. Commun. ACM 35, 10 (Oct. 1992), 83–91.
22. Baeza-Yates RA, Gonnet GH: A new approach to text searching. In Proceedings of the 12th International Conference on Research and Development in Information Retrieval Edited by: Belkin NJ, van Rijsbergen CJ. Cambridge, MA: ACM Press; 1989, 168–175.

23. Cédric Notredame, Desmond G Higgins, Jaap Heringa, T-coffee: a novel method for fast and accurate multiple sequence alignment¹¹ Edited by J. Thornton, Journal of Molecular Biology, Volume 302, Issue 1, 2000, Pages 205-217, ISSN 0022-2836
24. Huang, X Q et al. "A space-efficient algorithm for local similarities." Computer applications in the biosciences : CABIOS vol. 6,4 (1990): 373-81. doi:10.1093/bioinformatics/6.4.373
25. Kazutaka Katoh, Kazuharu Misawa, Kei-ichi Kuma, Takashi Miyata, MAFFT: a novel method for rapid multiple sequence alignment based on fast Fourier transform, Nucleic Acids Research, Volume 30, Issue 14, 15 July 2002, Pages 3059–3066
26. Edgar, Robert C. "MUSCLE: multiple sequence alignment with high accuracy and high throughput." Nucleic acids research vol. 32,5 1792-7. 19 Mar. 2004
27. Edgar, Robert C. "MUSCLE: a multiple sequence alignment method with reduced time and space complexity." BMC bioinformatics vol. 5 113. 19 Aug. 2004, doi:10.1186/1471-2105-5-113
28. Kimura, Motoo. The neutral theory of molecular evolution. Cambridge University Press, 1983.

29. Papadopoulos, Jason S, and Richa Agarwala. "COBALT: constraint-based alignment tool for multiple protein sequences." Bioinformatics (Oxford, England) vol. 23,9 (2007): 1073-9. doi:10.1093/bioinformatics/btm076
30. https://en.wikipedia.org/wiki/Sequence_logo (Last edited: 24.02.2022, Last visited: 18.01.2023)
31. Thomsen, Martin Christen Frølund, and Morten Nielsen. "Seq2Logo: a method for construction and visualization of amino acid binding motifs and sequence profiles including sequence weighting, pseudo counts and two-sided representation of amino acid enrichment and depletion." Nucleic acids research vol. 40,Web Server issue (2012): W281-7. doi:10.1093/nar/gks469